Exploring the Potential of LLM-Driven Opponent in Damath

Alicando, Gon Vincent
Jamen, Ramel Cary
Escasio, Edward Vincent

Supervised by
Malikey M. Maulana

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

May, 2025

A undergraduate thesis submitted in partial fulfilment of the requirements for the degree of B.S. in Computer Science.

Copyright ©2025 MSU - Iligan Institute of Technology

www.msuiit.edu.ph

Abstract

Large Language Models (LLMs) have become increasingly prevalent in recent years, significantly advancing the field of Artificial Intelligence (AI). These models have the capability to understand context and generate human-like responses, and are used in various applications such as text generation, code generation, informational chatbots, and gaming. In the gaming industry, LLMs are utilized for generating dialogues and behaviors for non-player characters (NPCs), assisting in game design and development, and even playing the games themselves. The use of LLMs as game-playing agents is a growing area in AI. Furthermore, AI has been widely applied in well-known board games such as Chess, Checkers, Othello, and Go, providing an opportunity to explore lesser-known areas of AI applications in board games. This paper proposed to apply an LLM as a Damath opponent; an LLM Agent revolving in Agentic workflow with prompt-to-code generation, tool calling and meta-programming. The evaluation included a comparison to our LLM Agent against two established baselines: a Random Choice Generator (RCG) AI and a MiniMax AI player with alpha-beta pruning. A Damath Game Engine was also developed to facilitate the interaction between the user and the LLM Agent, processing inputs/outputs, and updating the game state of the Damath Board. Additionally, a web application for the Damath game was developed to provide a user interface that connects the user to the game engine and the LLM Agent. The LLM Agent substantially outperformed the Random Choice Generator AI in 50 games, although it was outmatched when it played 50 games against the Minimax AI. Nevertheless, it successfully carried out a complete game of DaMath from start to finish, demonstrating the potential of LLM-driven agents for DaMath and similar board game environments.

Keywords: Artificial Intelligence, Large Language Models, Board Games, Damath, LLM Agent

Contents

List of Figures

List of Tables

List of Abbreviations

1

Research Description

Large Language Models (LLMs) represent a significant advancement in Artificial Intelligence (AI), using machine learning techniques to understand and generate human-like text based on given inputs. These models, built on deep learning architectures such as transformers, are trained on vast datasets to discern patterns and structures within languages, enabling them to perform tasks such as text generation, translation, summarization, and question answering (Minaee et al., 2024). Examples like Generative Pre-trained Transformer (GPT) illustrate the remarkable capabilities of these models in generating coherent and contextually relevant text across diverse applications (OpenAI et al., 2023).

LLMs have numerous applications, enhancing efficiency and innovation in various fields. In customer service, they power chatbots and virtual assistants, providing conversational interfaces and support (Shum et al., 2018). Content creation benefits from LLMs through automated generation of articles, reports, and creative writing (Gehrmann et al., 2019). These models also play a crucial role in translation services, search engines, and healthcare, assisting in tasks such as text conversion, improving search relevance, and analyzing medical records (Lee et al., 2020; Wu et al., 2016).

A particularly exciting application of LLMs is their use as game engines, where they generate dynamic narratives, dialogues, and in-game content, creating immersive and personalized gaming experiences (Ammanabrolu et al., 2020). Recent years have seen an explosive increase in research on LLMs and their applications in games. A survey by Gallotta et al. (2024) has explored the current state of the art across the various applications of LLMs in and for games. The survey highlights the different roles LLMs can take within a game, from generating game content to facilitating communication between human players. The use of LLMs in games has also been

explored in various papers, such as "Human-Level Play in the Game of Diplomacy by Combining Language Models with Strategic Reasoning" (2022) and "GameEval: Evaluating LLMs on Conversational Games" (2023).

1.1 | Background of the Study

Recent advancements in Artificial Intelligence (AI), particularly through Large Language Models (LLMs), have demonstrated their versatility beyond traditional language tasks. LLMs, such as those based on the GPT architecture, are increasingly being explored for their potential in gaming, mostly as game-playing agents. This exploration extends beyond mere gameplay to encompass the underlying mechanics of various genres, including puzzle games and board games.

The study of LLMs as game-playing agents represents a significant departure from their original design intent of Natural Language Processing (NLP). Researchers have begun to investigate how these models can simulate and interact with complex game environments, leveraging their ability to generate coherent and contextually appropriate responses. For instance, in puzzle games, LLMs can attempt to solve intricate challenges by understanding clues, deducing patterns, and making informed decisions based on the rules provided.

Board games, with their structured rules and strategic depth, pose another intriguing challenge for LLMs. These models can potentially simulate entire game sessions, playing against human or AI opponents, and adapt their strategies dynamically based on the evolving state of the game. Moreover, LLMs could serve as virtual game masters, generating scenarios, providing guidance, and enhancing the overall gaming experience through natural language interactions.

In educational board games such as DaMath, there is a potential to integrate AI with education. Through a game-based learning approach, it has been found to improve the conceptual understanding of math students (Dalidig, 2020). Research indicates that game-based learning models, assisted by fun card puzzles, can significantly improve students' understanding of math concepts (Nuha and Purwanti, 2023). It has also been demonstrated that game-based learning improves learning outcomes by increasing student motivation and engagement in mathematics (Dalidig, 2020). Since DaMath is a checkers game combined with mathematical operations, it can enhance students' arithmetic skills by making learning math engaging and interactive (Bantilan et al., 2014). DaMath, created in the 1970s by Jesus Huenda in the Philippines, evolved to

incorporate more complex mathematical concepts, becoming a staple in Filipino schools and promoting mathematical literacy and critical thinking (Lee-Brago, 2010). Together, LLMs and educational games like DaMath underscore the potential for technological and educational synergy to enhance learning outcomes and foster greater student engagement.

In summary, while LLMs have primarily been celebrated for their prowess in language tasks, their application as an LLM-Driven opponent introduces a novel dimension to AI research and development. Understanding their capabilities and limitations in gaming contexts will pave the way for future advancements in both AI-driven entertainment and educational technologies.

1.2 | Statement of the Problem

Despite the widespread use and development of Large Language Models (LLMs) in various domains such as content generation, virtual assistants, and search and code development, one area that has received relatively little attention is the integration of LLMs within games. In addition, AI applications in DaMath are limited, especially with regard to LLM, since there are no known existing studies on the use of LLM in DaMath.

1.3 | Objectives

The subsequent sections outline both the general and specific objectives of this study.

1.3.1 | General Objective

The main objective of the study is to explore the utilization of LLM as a game-playing agent capable of playing against real players on the DaMath board game.

1.3.2 | Specific Objectives

1. To develop a board structure and piece movement representation of DaMath to serve as an input to the LLM Agent.

2. To create the game engine that is responsible for the input and output of user and LLM Agent, and vice versa.

3. To create an LLM Agent designed specifically to play DaMath that utilizes an instruct-based Large Language Model, structured with a sequence of instructions used in determining the pieces' valid moves to deciding its best move.

4. To evaluate the LLM Agent's game performance against a Random Choice Generator (RCG) AI Player and a Depth-Limited Alpha-Beta Pruning MiniMax AI Player.

5. To create a prototype of the DaMath system incorporating the game client and game server that is integrated with an LLM Agent.

1.4 | Scope and Limitation

This study focused on the development of an LLM Agent capable of playing against human players in the game DaMath. The LLM used is a pre-trained instruct-based model, so the performance of the DaMath Engine relies on the performance of the LLM itself. Existing LLM models are prone to hallucination; therefore, mitigating this issue involved prompt testing to assess and improve performance. The DaMath game developed did not prioritize time constraints, as it focuses on enhancing the player's learning experience rather than the competitive aspect of the game.

1.5 | Significance and Contribution

This research holds significance in the realm of Game Design and Development, as it involves creating a DaMath game and developing an LLM Agent tailored for this game. The study aims to explore the model's capability in the challenging process on creating the aforementioned LLM Agent. By leveraging a Large Language Model as an agent for the game DaMath, this research could potentially set a standard for AI applications. Moreover, it aims to provide valuable insights for future integration of LLMs as a game-playing agent.

2

Literature Review

This chapter compiles information from various books and journals written by experts in the field. It provides a foundation for understanding existing studies and identifying gaps in current research relevant to the study's focus. The following sections will explore key insights, including different approaches to gaming with Artificial Intelligence and the applications of Large Language Models.

2.1 | AI on Board Games

The rise of artificial intelligence (AI) over the years has significantly influenced the advancement of modern technology. Artificial Intelligence is applied to various fields with few examples such as healthcare, education, social media, and games, specifically board games. A board game is a type of game which is played by placing or moving pieces on a board (Merriam-Webster). By employing advanced techniques in the application of AI to board games, it is now possible for an AI computer player to compete with human players or even to defeat the most skilled opponents.

A very known example of this is the IBM Deep Blue (or the Deep Blue II), a chess machine developed at IBM Research during the mid-1990s (Campbell et al., 2002). The IBM Deep Blue was able to defeat an old World Chess Champion Garry Kasparov. Another example is a simple algorithm for an AI computer as an opponent to a Tic-Tac-Toe game (Karamchandani et al., 2015). In the following subsections, more AI applications on board games will be presented, including the aforementioned examples.

The various works will be classified into simple AI algorithms and higher-complexity AI applications. The works that will be mentioned in the first

subsection will contain relatively simple AI applications to board games such as the usage of fuzzy logics, a*, minimax/alpha-beta pruning minimax, evolutionary algorithms, and more. In contrast, the works that involve higher-complexity AI applications such as any type of neural network implementation will be included in the next subsection.

2.1.1 | Simple AI Algorithms

The application of simple AI algorithms has been used in different board games, especially the simplest game of Tic-Tac-Toe. Karamchandani et al. (2015) proposed a simple algorithm for an AI computer as an opponent in Tic-Tac-Toe. The algorithm comprises the usage of fuzzy logics, as well as decision-making based on multiple conditional cases. The developed algorithm is purposefully filled with certain open areas which gives players the opportunity to win against the computer. However, the proposed algorithm was very hard to follow through especially since it was filled with a lot of if statements to cater to the different conditional cases in the game's different states. On a much higher level of complexity compared to the usage of fuzzy logic, the study of Zain et al. (2020) introduces a heuristic search approach in Tic-Tac-Toe using minimax algorithm, specifically an optimization of the minimax algorithm which is alpha-beta pruning. This works by generating a tree of possible moves, and choosing a move that minimizes the maximum possible loss. This ensures that the computer will choose the best moves that it can find. By implementing an alpha-beta pruning algorithm, the search will be limited, resulting in faster computations. This is done by cutting off branches of game trees that are not needed for further searching when there is a better movement that is already available. This method is also used by Sahu et al. (2012), although they introduced different levels of difficulty. On the easiest level, a random selection algorithm is used for the computer's moves. It then generates a random move, and also makes sure that the space for the move is not taken yet. For the hard difficulty, it checks for win/lose conditions and then performs a move based on the multiple conditions. However, if no conditions are satisfied, it utilizes the random selection algorithm for its moves. The hardest difficulty is an alpha-beta pruning optimization of a minimax algorithm, similar to the previous study.

In the study of Escandon and Campion (2018), The concept of minimax algorithm is used to develop a GUI for a vs-computer Checkers game. In their study, they also evaluated the minimax algorithm by the duration that it takes for the computer to perform a minimax search depending on the depth. It is shown that in the depths 3 and

4, the runtime is increased very slightly, while significant increase is observed in depths 5 and 6. Similar to Sahu et al. (2012)'s AI implementation in Tic-Tac-Toe, a study by Idzham et al. (2020) introduces different difficulty levels on their computer opponents for checkers. It still utilizes an alpha-beta pruning minimax algorithm, but they introduced 2 levels: Novice and Optimal AI. They work similarly, with the difference being the novice implementation dividing 2 to the next minimax value, opting an unfit movement compared to the optimal movement that the second implementation will get. A genetic algorithm version of an AI for checkers was created by Barros et al. (2011). Their goal was to provide an opponent for players to improve their abilities. Their GA implementation only applies the mutation operator to their population. The mutation that they used has two types; mutation by sub-exchange and mutation by replacement. Mutation by sub-exchange switches 2 elements of a chromosome, and returns the new chromosome. The mutation by replacement chooses 2 different elements of the chromosome and replaces the values with their corresponding complements in a preset alphabet.

Most of the studies mentioned are implementing an alpha-beta pruning minimax algorithm, but alpha-beta pruning only achieves optimal results if an adequate evaluation function exists, and the game has a low branching factor (Chaslot et al., 2021). With this in mind, Chaslot et al. (2006) proposed an Objective Monte-Carlo, which is a Monte-Carlo Tree Search (MCTS) for the board game Go. It is a best-first search algorithm that builds and uses a tree of possible future game states, which utilizes these mechanisms: Selection, Expansion, Simulation, and Backpropagation. MCTS provides a better alternative to alpha-beta pruning for other board games such as Go, modern board games, and video games.

On other various works, a study by Konen (2019) presented a new general board game (GBG) framework for learning and playing. Game developers (specifically board games) can follow certain interfaces described in the GBG framework to be able to use their built-in AI agents, as well as implement new AI agents easily. Another study from Crha (2020) introduces a multi-player board game with imperfect information states called Colonizers. They also created a framework for easier addition of new AI algorithms. In the paper, they used 4 algorithms for different AI agents: random decisions, heuristics, MaxN, and Information Set Monte-Carlo Tree Search (ISMCTS). The random decision AI is straightforward, picking a random action on a given action set. The heuristic AI performs an action on a set of conditional cases, returning to picking a random action if there are no conditions that are applicable. MaxN uses minimax techniques but intended for games with more than two players. Unlike

minimax that is applicable mainly to zero-sum games, MaxN prioritizes maximizing the gain of each player. ISMCTS is a type of MCTS that is used in games with imperfect information, which is fitting for the developed board game. The best performing AI agent is the ISMCTS, overwhelmingly dominating the other AI agents.

2.1.2 | Higher-complexity AI Applications

Various works that involve relatively higher-complexity AI applications in board games will be discussed here. These include utilizations of any forms of neural networks, as well as other high-complexity AI compared to the previous subsection.

A study conducted by Ling and Lam (2011) presents an algorithm for the game Tic-Tac-Toe, which is then learned by a neural network with double transfer functions (NNDTF), and then trained using an improved Genetic Algorithm (GA) which introduces new genetic operators. The proposed neural network exhibits a node-to-node relationship in the hidden layer which greatly improves the learning capabilities of the network. It is then tested against a traditional 3-layer feed-forward NN which is also trained by GA, that also utilizes the algorithm presented by the researchers. The proposed algorithm performed better than the traditional one, indicating that the application of a node-to-node relationship impacted the performance of their neural network. Another study of Inan et al. (2021) developed a Tic-Tac-Toe agent by integrating a tree-based supervised machine learning (ML) XGBoost, in a probabilistic manner along with rule-based inference. Various ML models were tested against a minimax algorithm for Tic-Tac-Toe, but only XGBoost performed the best, resulting in a draw for every match against the minimax algorithm.

Additionally, the approach of an evolutionary algorithm to a neural network is applied in Othello to discover complex Othello strategies in the study of Moriarty et al. (1995). There are two strategies presented, which are positional and mobility strategies. A positional strategy emphasizes on taking specific positions and piece configurations in the board, such as the corners and edges. On the other hand, a mobility strategy still emphasizes on the corners, but not on the other things that the positional strategy focuses on. The mobility approach focuses on forcing the opponent to make moves to surrender a corner, which in turn allows you to take that corner. The proposed game-playing AI is then made to compete with a random mover, which in time developed a positional strategy. The networks that won were chosen and then tested against an alpha-beta pruning minimax, which resulted in a very poor gameplay initially. After the population evolved over time, it presented a different approach from

the positional strategy, and started to win against the alpha-beta algorithm.

A checkers AI was implemented by Chellapilla and Fogel (1999) through using an evolutionary algorithm (EA) to train several neural networks, which serves as an evaluation function for an alpha-beta minimax. With the model's lack of expert knowledge in the game, and the board configuration being a vector of length 32, the model lacks the recognition of the spatial characteristics of the board. However, it manages to develop a newfound knowledge by repeated training. The proposed model performed very well, managing to defeat several expert-level players, and also managed a draw against a master.

DeepChess is a chess-playing program that does not have any a priori knowledge of chess, but its performance is better than chess engines such as Crafty and Falcon (David et al., 2016). As a matter of fact, DeepChess performs on par with Falcon with DeepChess performing 4 times slower. By allowing DeepChess to use more time than Falcon, it was able to dominate over Falcon. DeepChess is trained on a large dataset of chess positions. It starts by using deep unsupervised NNs for pretraining, then training a supervised NN. The supervised NN is then used in a new form (Comparison-Based) Alpha-Beta Pruning Search. After that, the inference speed is then improved by network compression or distilling.

Most of the works mentioned mainly use some sort of a minimax algorithm. However, the minimax algorithm falls short in multiplayer games that are greater than two, as well as non-zero-sum games. In a study by Brown and Sandholm (2019), they introduced Pluribus, a superhuman AI for multiplayer six-player no-limit Texas Hold'em Poker. Pluribus applies various different AI applications, such as exploring strategies through self-play, applying depth-limited search for efficient computation, Monte Carlo Counterfactual Regret Minimization (MCCFR) which minimizes regrets for actions not taken in counterfactual scenarios, and more. Pluribus was able to perform greatly with 5 copies and 1 human playing. It also dominated 5 humans and 1 Pluribus, over the course of 10,000 hands.

2.2 | Large Language Model on Games

LLMs are highly prevalent in today's generation, revolutionizing various industries by enhancing natural language understanding and generation. Early language models, known as Statistical Language Models (SLMs), relied on probabilistic techniques to predict the next word in a sequence based on the previous words, these models

were limited by their reliance on statistical methods (Ronald, 2001). The advent of Neural Language Models (NLMs) marked a significant improvement, as they leveraged neural networks to capture more complex patterns and enhance prediction accuracy. Subsequently, Pre-trained Language Models (PLMs) emerged, benefiting from extensive pre-training on large datasets, which allowed them to learn a broad range of language patterns before being fine-tuned for specific tasks. The most advanced stage in this evolution is represented by Large Language Models (LLMs), which boast a massive number of parameters and are capable of understanding and generating human-like text with remarkable accuracy (Chu et al., 2024).

Fine-tuning a Large Language Model (LLM) involves pre-training on large datasets and instruction-tuning to improve task-specific performance. Human feedback is used to create a reward model that evaluates the LLM's outputs. Through Reinforcement Learning from Human Feedback (RLHF), the LLM is iteratively updated, aligning it with human preferences and improving its effectiveness in generating desired outputs. Recent advancements in LLMs are mainly driven by data diversity, with its extensive internet-based sources that enriches training data, improving generalization and multitasking (Naveed et al., 2023). Given its capabilities in general-purpose language generation, gaming made possible utilizing LLMs as a Game Engine, recent study of Gallotta et al. (2024) enumerates the role of LLMs in Games; (1) as a Player, (2) Non-Player Character (NPC), (3) Player Assistant, (4) Game Master, (5) Game Mechanic, (6) Automated Designer. These gaming roles will be elaborated in the following sections below:

2.2.1 | Game playing

An LLM can function as a player within the game, taking the place of a human player and emulating their objectives. A study of cooperation and coordination behavior of LLMs (GPT -3, GPT-3.5, and GPT-4) by Akata et al. (2023), advances the understanding of LLM social behavior and suggests a foundation for machine behavioral game theory. The LLMs plays finitely repeated conversational games (Prisoner's Dilemma and Battle of the Sexes) with each other and with human-like strategies. Findings is that LLMs generally perform well in these tasks, GPT-4 excel in self-interest games such as the iterated Prisoner's Dilemma but struggle with coordination games like the Battle of the Sexes; unless given more information about other players or by predicting their actions. Additionally, Xu et al. (2024) investigated a well-known communication game "Werewolf" in their study, concluding LLMs (GPT-3.5-turbo-030)

exhibits non-preprogrammed emergent strategic behaviors such as trust relationships, confrontation, camouflage, and leadership. Similarly, from Tsai et al. (2023) delves into the world of adventure text-based game with Zork I, testing a language model and other state-of-the-art systems with goal-based and decision making during the gameplay.

Another interesting roles are being a Non-Player Character (NPC), like an adversary or conversational partner and as an assistant providing hints or handling menial tasks for a human player. Traditionally, NPCs have been restricted to predetermined scripts, leading to repetitive dialogues that neither interact with nor influence players' strategies. In contrast, LLMs offer the potential for creating NPCs with human-like interactions, capable of responding to player behaviors. For instance, Kumaran et al. (2023) introduce SceneCraft, a narrative generation tool for 3D games based on LLMs. SceneCraft not only manages dialogues but also generates NPC emotions and gestures during player interactions. As an assistant, in The Sims, a disembodied assistant offers context-specific tips through dialogue boxes. Similarly, Civilization VI features various visually represented assistants that provide advice based on their unique heuristics, suggesting optimal build options and potentially easing the player's decision-making process (Gallotta et al., 2024).

It was found that large language model (LLM) players excel in three broad categories of games: (a) those where game states and actions can be efficiently represented as sequences of abstract tokens, (b) those that primarily use natural language for input and output, and (c) those that allow external programs to manage player actions through an API (Gallotta et al., 2024). In this context, board games fits to those said criteria; with its discrete board positions, moves that are easily represented in formats like Portable Game Notation (PGN) and action selection problem where it aligns with the autoregressive learning objective of LLM, making board games good for exploration. This subject matter is thoroughly discussed in section 2.3.

2.2.2 | Game Master

As a Game Master role controlling the flow of the game, or hidden within the games' ruleset (controlling a minor or major mechanic of the game), and algorithmic generation of game content such as levels, visuals, or even entire game. This was made possible through LLMs' recent implementation of Procedural Content Generation (PCG) technique that aims to generate game content using computer software (Summerville et al., 2018). Recently, an initiative of IEEE Conference on Game, the Chatgpt4PCG competition focuses on leveraging ChatGPT to create game levels for

Angry Birds (Taveekitworachai et al., 2024). Similarly, Sudhakaran et al. (2023) with their MarioGPT explored the Super Mario Brothers game with generating infinite level stage via prompts. In the same context, another 2D game "Sokoban" also been explored by Todd et al. (2023) with their fine-tuned LLMs to generate novel and playable Sokoban levels. Studying level generation with ChatGPT can assist researchers in gaining a deeper understanding of the strengths and limitations of LLMs. Moreover, a higher complexity study of Hu et al. (2024) with their exploration in proposing a Text to Building in Minecraft (T2BM) model that refines user inputs, encodes buildings by an interlayer and fixes errors via repairing methods. Inspired by above-mentioned game generation through PCG ideas, Zhao et al. (2024) proposed an LLM-based framework to generate game rules and levels simultaneously based on Video Game Description Language (VGDL).

There are however other roles an LLM can play outside of the game's runtime, such as a designer for the game (replacing a human designer). An initiative of collaborating LLMs as Evolutionary Engines by Lanzi and Loiacono (2023) finds that, by leveraging the generative capabilities of these models, the game design process can be significantly enhanced. In a similar context, a study of Nasir et al. (2024) "Word2World", a system that allows LLMs to generate playable games through storytelling without requiring task-specific fine-tuning. Word2World harnesses LLMs' capabilities to produce varied content and extract information. By integrating these skills, LLMs can craft a game's story, design its narrative, and strategically place tiles to construct coherent worlds and playable games. Additionally, LLMs can generate a multitude of creative concepts for game mechanics, storylines, and character designs, providing a rich foundation for human designers to refine and develop further. This iterative process, where ideas evolve and improve over successive generations, mirrors evolutionary principles, leading to increasingly sophisticated and engaging game designs (Gallotta et al., 2024).

2.3 | Large Language Model on Board Games

Board games are among the domains where Large Language Models (LLMs) have shown notable progress. The use of LLMs in board games has created new opportunities for player interactions, game strategy analysis, and the creation of intelligent gaming systems (Gallotta et al., 2024). Large Language Models (LLMs) have been applied to board games like Tic-Tac-Toe in order to evaluate players' strategic decision-making ability. The study by Topsakal and Harper (2024) examined how well LLMs performed in a Tic-Tac-Toe game, demonstrating their ability to understand the rules and make

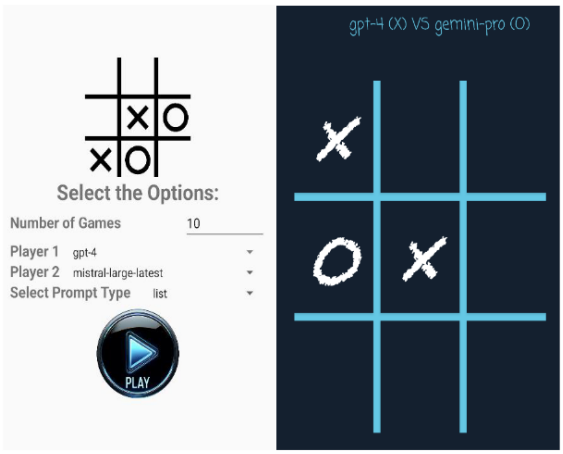


Figure 2.1: (Left) the screen to select the number of games, players, and the prompt type. (Right) the screen displays how the game progresses (Topsakal and Harper, 2024).

calculated plays in a supervised environment.

To provide another insight into how language models can learn and utilize internal representations in a controlled gaming environment, the study by Li et al. (2024) discussed another GPT variant named Othello-GPT that is trained to predict legal moves in the game of Othello, it examined the internal representations that the model learned without having to know the game's rules beforehand. An 8-layer GPT model with an 8-head attention mechanism and a 512-dimensional hidden space was trained using sequential tile indices as input. The model was trained in an autoregressive manner and became highly accurate at predicting legal moves. A related study by Toshniwal et al. (2021) that investigated language models trained on chess move sequences was also cited in the study. In this work, typical language modeling architectures were examined in a well-defined, confined environment. The study discovered that these algorithms were highly accurate in predicting legal chess moves. The study also demonstrated that the model seems to track the board condition by examining projected moves. In their investigation, the authors did not, however, go into detail about the structure of any internal representations.

LLMs use enormous online datasets to play more complicated board games like Go and Chess. Their ability to comprehend past game trends and make well-informed decisions is enhanced by this abundance of knowledge. According to (Guo et al., 2024), LLMs serve as sophisticated tools for decision-making, helping to focus possible courses of action and effectively assess their strategic ramifications. When evaluating game states and the effects of different plays, they can effectively take the place of standard

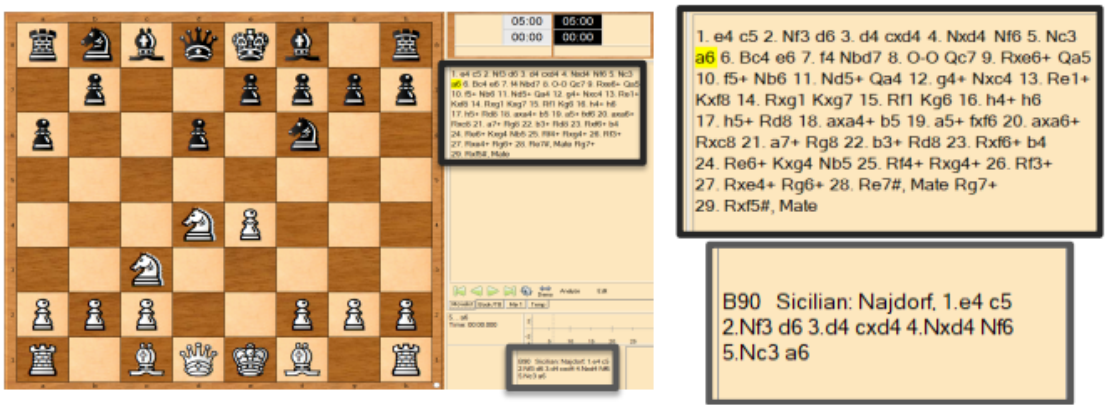


Figure 2.2: Generated GPT-2 example games. Every Transformer game that is created is recorded on video, divided into individual moves for analysis, and inputted into Portable Game Notation (PGN) with both automated strategy notations and moving pieces. (Noever et al., 2020).

value functions. Furthermore, without requiring further training, LLMs can further expedite decision-making processes by combining with algorithms such as Monte-Carlo Tree Search (MCTS). Another study by (Noever et al., 2020) focuses on the "Chess Transformer," provides an example of how LLMs can be refined on large-scale game data datasets (in this case, 2.8 million chess games in Portable Game Notation). This method shows how model design and attention mechanisms contribute to the model's ability to develop realistic tactics and identify classic game openings, demonstrating its strategic modeling proficiency.

A similar study by Ciolino et al. (2020) looks into the use of GPT-2, a type of natural language modeling, to generate strategic moves in the game of Go. The model, called the "Go Transformer," improved GPT-2 by adjusting it using a dataset of 56,638 Go games in Smart Game Format (SGF). This allowed the machine to learn to produce logical player formations and plays. During training, the preprocessed dataset considers each game as a single line to increase readability. The Go Transformer successfully replicates normal SGF syntax and grid intersections in game sequences, as demonstrated by the results. The logical forms and sequencing of the model are validated using visualizations utilizing the Sabaki project. The study shows how language models such as GPT-2 can be used to generate and recognize patterns in gaming issues.

A study about a board game called Catan by Martinenghi et al. (2024) looks at how well large language models (LLMs) categorize dialogue acts (DAs) in multi-party game

conversations. It looks on how different prompts, the length of the context, and the order of the dialogue affect LLM performance. The findings indicate that while giving more DA instances improves performance, giving dialogue last decreases accuracy. Longer contexts may reduce precision but increase memory and F1 scores. The study highlights how crucial dialogue sequence and context duration are to prediction accuracy. It also contrasts human and LLM pragmatic processes, pointing out that LLMs tend to rely on shortcuts that have an impact on comprehension in multi-party dialogues. The report identifies some areas for improvement and urges additional research into LLM performance in complicated interactions.

2.4 | DaMath

DaMath is an educational board game originating from the Philippines, blending the traditional Filipino game "dama" with mathematical concepts, hence its name. It was invented by Jesus L. Huenda, a public high school teacher in Sorsogon, who encountered difficulties in teaching math using traditional methods. Inspired by a student project called “Dama de Numero” submitted by Emilio Hina Jr. in 1975, Huenda overhauled the game and introduced it to his class, who enjoyed playing it until it is commonly played in all elementary and secondary schools in the Philippines (Wikipedia, 2024). Huenda’s innovative approach has been recognized nationally and was awarded in 1981 with presidential merit medal, and the game has equipped Filipino students with essential numeracy skills. DaMath encourages strategic thinking and problem-solving through creative activities like storytelling, flowcharts, and simulations (Dep-Ed, 2010).

2.4.1 | Game Setup

The board consists of 64 squares arranged in a pattern of alternating black and white, similar to a chessboard. Figure 2.3 illustrates that the four fundamental mathematical operations are positioned on the white squares. Each square is labeled using a notation indicating its column and row. For instance, the square at the top-left corner is located at column 0 and row 7, identified as (0, 7).

The figures 2.5, 2.6, 2.7, 2.8 shows the counting numbers, whole numbers, fraction and integer variations with their respective piece positions and value.

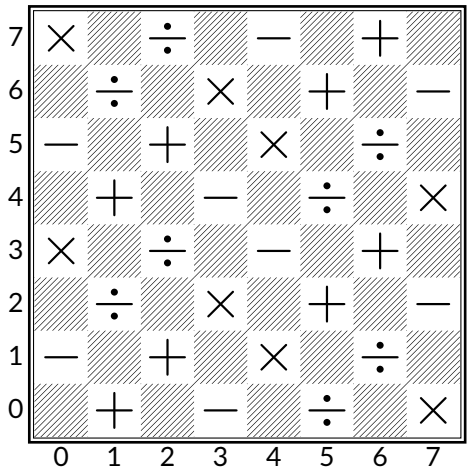


Figure 2.3: The DaMath Board (Math and Multimedia, 2016)

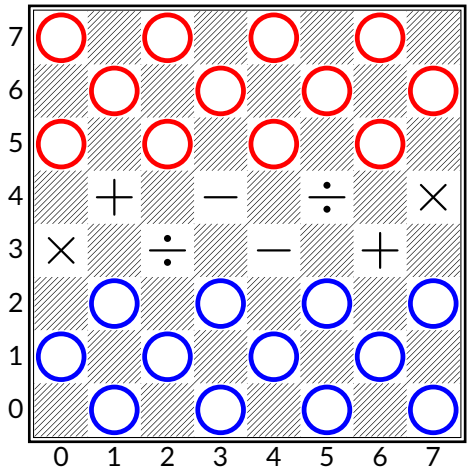


Figure 2.4: The DaMath Board: Piece Positions (Math and Multimedia, 2016)

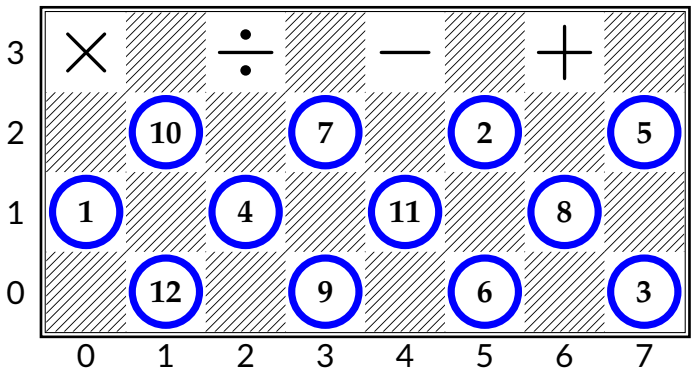


Figure 2.5: DaMath: Counting Numbers (Gal, 2017)

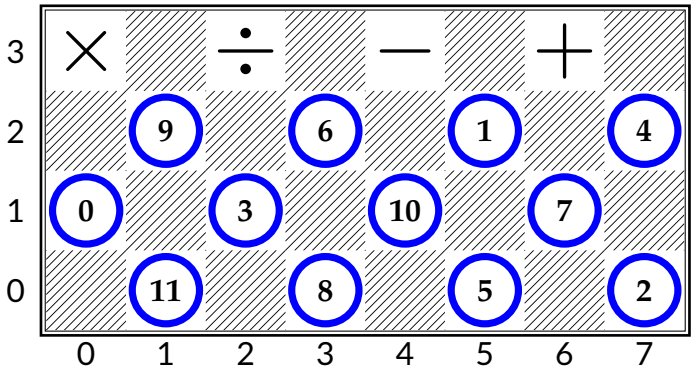


Figure 2.6: DaMath: Whole DaMaths (Gal, 2017)

2.4.2 | Game Mechanics

Since DaMath is derived from the traditional game of Dama, it follows some of its basic mechanics but introduces unique twists to incorporate mathematical elements. The following are its standard mechanics:

Earning the "Dama" Chip: A dama chip is an upgraded piece that provides an advantage by moving through multiple squares. A player's chip becomes a dama when it reaches the opponent's squares (1,0), (3,0), (5,0), or (7,0). Similarly, the opponent's chip becomes a dama upon reaching the player's squares (0,7), (2,7), (4,7), or (6,7).

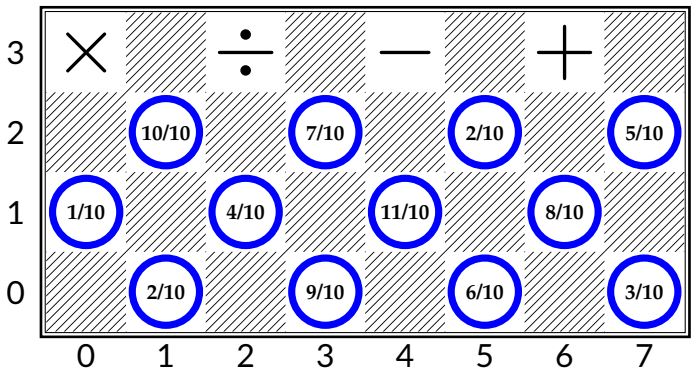


Figure 2.7: DaMath: Fraction DaMaths (Gal, 2017)

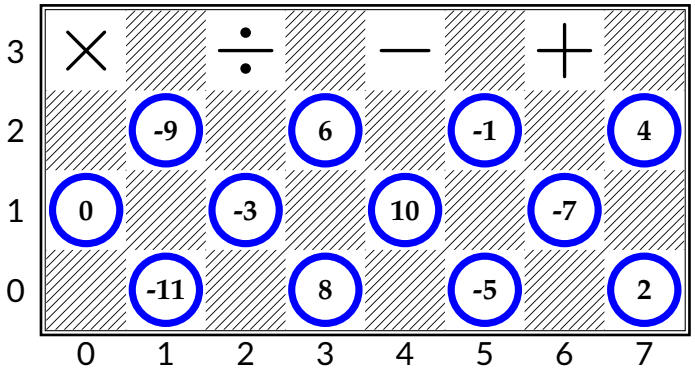


Figure 2.8: DaMath: Integer DaMaths (Gal, 2017)

Moving the Chips: Players can only slide their chips diagonally forward into an empty adjacent white square. Backward slides are not allowed unless capturing an opponent's chip. Players take turns moving a chip, and passing a turn is not permitted. The "touch move" rule is in effect, meaning once a player touches a chip, they must move it. Each player has 60 seconds per turn to make a move. If the chip is "Dama", it can move diagonally long forward or backward to any empty square until it reaches an allied chip.

Capturing an Opponent's Chip and Scoring if Ordinary Chip: If an ordinary chip is to capture an opponent's chip, a player must "jump" over it diagonally within a diagonally adjacent tile next to the piece that is about to be taken, landing on the empty white square adjacent to the captured chip. Captures can be made diagonally forward

Player A (FIRST TO MOVE)			Player B		
Move	Score	Total	Move	Score	Total
R.C.			R.C.		
Total Score			Total Score		

Figure 2.9: Sample of DaMath Scoring Sheet (Fabula, 2022)

or backward, and players cannot deny a capture. Upon capturing a chip, the capturing player scores points based on the mathematical operation indicated on the white square where the chip lands. This operation is applied to the numbers on the capturing and captured chips.

Capturing an Opponent's Chip and Scoring if Dama Chip: If the chip is "Dama", they capture opponent chips by jumping over them diagonally no matter how long the distance between the two chips is, as long as there is a vacant space behind the chip that is about to be taken. The score from the dama chip's capture is calculated by doubling the result of the mathematical operation indicated on the landing square. The score is also doubled if an ordinary chip captures a dama chip and if a dama chip captures another dama chip the score would be quadrupled.

Concluding the Game: The game concludes after 20 minutes or if a player cannot move any chips because they are "trapped." The final score is the sum of points from capturing opponent chips and the numbers on remaining chips on the board, with dama chips' scores doubled. The player with the highest final score wins.

2.4.3 | AI Implementations and Applications

A study by Ramos et al. (2013) presented a Mobile DaMath, addressing accessibility issues to students or users eager to play DaMath. Their application supported player-to-player gameplay, as well as player-to-computer. Their AI computer was constructed using an alpha-beta pruning minimax algorithm. As their focus was to create a mobile DaMath game, there were limited remarks about the AI opponent. The AI played well, but it performed predictable moves most of the time.

Another study by Maulana (2019) introduced a learning agent capable of playing DaMath, using a feed forward neural network with temporal difference learning or TD(λ). Different learning agents were trained by self-play for 500,000 games, and played against a minimax player choosing a random move from the top 3 best moves. Considering that there were no data of DaMath games used in the training of the learning agents, agents with lookahead performed averagely compared to agents without lookahead, with a lower performance.

2.5 | Summary

This chapter presents numerous studies that integrates the concept of Artificial Intelligence to games, with the focus being the application of LLMs in board games. Simplistic algorithms and neural network approaches have been applied to relatively-simple board games such as Tic-Tac-Toe, to higher-difficulty games such as Chess.

In the usage of LLM, a GPT model as a game-playing opponent in the game Othello was trained by using datasets on past games or existing game transcripts (list of moves by players). Without any a priori knowledge on the game itself, it was able to perform well and it was shown that their model was showing evidence of an emergent nonlinear internal representation of the board. Another LLM implementation is in testing the language models' next-token prediction using various tasks by providing a sequence of chess moves followed by a starting square movement or a piece. These models are trained with varying datasets, and models trained with enough training data learned to track pieces and predict legal moves, even if it was only trained on move sequences. For small training sets, by providing access to board state information, the model yielded significant improvements. Another utilization of LLM in the board game Catan was also presented, however it has complex rules involving resource management and trading, while chess has a simpler set of movement rules. Other various LLM applications in

board games were shown, however they involve training with large datasets.

In the context of DaMath, there are no known existing datasets for previous DaMath games conducted, so training LLMs in that way is out of the question. However, similar to one study involving testing the LLM's next-token prediction with one of the models having a small training set, instead of relying on previous conducted games in DaMath it can be possible to provide access to the board game state instead. With this, instead of training an LLM with datasets of DaMath games, an approach of creating an LLM-driven opponent in DaMath can be achieved by utilizing a pre-trained LLM, which is used by an Agent tailored with instructions and knowledge base in DaMath (game rules and game mechanics). The problem now lies in how the structures in the DaMath game are presented in such a way the LLM Agent can understand.

Table 2.1: An overview of Artificial Intelligence and Large Language Model application in gaming involving board games

Authors	*Approach*	*Methodology*	*Board Game*	*Implementation*
Karamchandani et al., 2015	Algorithm	fuzzy logic & conditional cases	Tic-Tac-Toe	Game AI
Zain et al., 2020	Algorithm	heuristic evaluation function + alpha-beta pruning	Tic-Tac-Toe	Game AI
Escandon and Campion, 2018	Algorithm	minimax	Checkers	Game AI
Barros et al., 2011	Algorithm	modified genetic algorithm	Checkers	Game AI
Chaslot et al., 2006	Algorithm	objective monte carlo (MCTS)	Go	Game AI
Konen, 2019	Framework	General Board Game framework	multiple board games	Framework
Crha, 2020	Algorithm	randomizer, heuristics, MaxN, ISMCTS	Colonizers	Game + Framework + Game AIs
Ling and Lam, 2011	Neural Networks	NNDTF + GA	Tic-Tac-Toe	Game AI
Inan et al., 2021	Machine Learning	tree-based supervised ML + rule-based inference	Tic-Tac-Toe	Game AI
Chellapilla and Fogel, 1999	Neural Networks	NNs trained by EA used in minimax	Checkers	Game AI
David et al., 2016	Neural Networks	un/supervised NN training + alpha-beta + network distilling	Chess	Game AI
Brown and Sandholm, 2019	Various Techniques	self-playing, DLS, MCCFR, etc	Poker	Game AI
Ramos et al., 2013	Algorithm	alpha-beta pruning minimax algorithm	DaMath	Game AI
Maulana, 2019	Neural Networks	feed forward NN w/ TD(λ)	DaMath	Game AI
Ciolino et al., 2020	Generative AI	LLM - FT-GPT2 as Transformer	Go	Pattern Search + Game Move Generator
Noever et al., 2020	Generative AI	LLM - FT-GPT2 as Transformer	Chess	Game Move Generator + Live-playing
Martinenghi et al., 2024	Generative AI	LLMs - Zero and Few-shot Learning	Settlers of Catan	Game-Based Conversations + Dialogue Act Classification
Li et al., 2024	Generative AI	LLM - Othello-GPT + Probes + Interventional Techniques	Othello	Predicting Legal Moves
Toshniwal et al., 2021	Generative AI	LLM (GPT2) - Probes (RAP)	Chess	Board State Probes + Benchmarking LLMs
		Our Proposed Solution		
Alicando et al.	Generative AI	LLM (LLaMa) - Agent	DaMath	LLM-Driven Opponent

3

Theoretical Framework

This chapter discusses the key concepts and the relative concepts applied in achieving the goal of an LLM-Driven opponent.

3.1 | Agents

LLM agents are AI-powered systems that use large language models to interpret input, engage in conversation, and perform tasks by following a structured, goal-driven workflow. Acting as the central "brain" of an application, an LLM agent coordinates sequences of actions to fulfill user requests or solve problems. This agentic behavior enables context-aware reasoning, dynamic decision-making, and integration with tools or APIs (Chudleigh, 2025).

3.1.1 | Langchain

The LangChain framework enables this by streamlining the development and deployment of LLM-based applications, supporting scalable and context-aware solutions through features like code generation, output parsing, tool integration, and secure API management (Mavroudis, 2024). Additionally, LangSmith, a companion platform for building production-grade LLM applications, provides powerful observability, evaluation, and prompt engineering capabilities: it enables detailed analysis of execution traces and the configuration of metrics, dashboards, and alerts; supports running evals over production traffic with scoring and human feedback; and facilitates prompt iteration with automatic version control and collaboration features. These features ensure teams can ship LLM applications quickly and with confidence (see Figure 3.1).

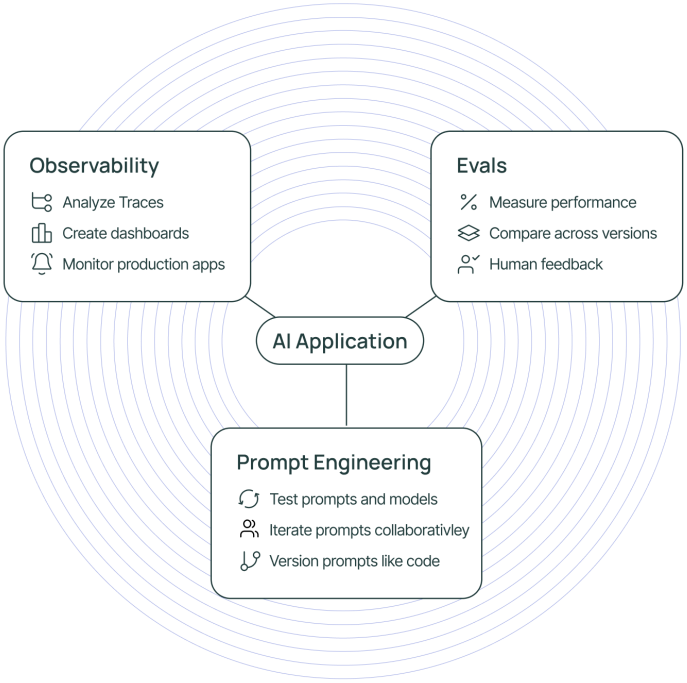


Figure 3.1: LangChain Architecture

3.1.2 | LangSmith

LangSmith is a companion platform to LangChain that helps monitor, debug, and evaluate LLM applications. It provides tools for tracing agent workflows, analyzing prompts, and collecting feedback to improve performance. With built-in evaluation metrics and version control, LangSmith supports prompt iteration and testing (Ito et al., 2020).

3.1.3 | Language Model

We employ two versions of the LLaMA3 model family: `llama3.1:8b-instruct-fp16` and `llama3.2:3b-instruct-fp16`. These advanced instruct-based language models, with 8 billion and 3 billion parameters, respectively, are utilized to power the core capabilities of our system. LLaMA3 models are renowned for their state-of-the-art natural language processing performance, delivering high levels of accuracy, fluency, and efficiency in generating human-like text (AI@Meta, 2024). To deploy and manage these models effectively, we utilize the Ollama platform, an open source system that allows for the seamless running of large language models on local machines. Ollama

provides robust infrastructure support, including scalability, secure local data handling, and user-friendly model management interfaces. Together, LLaMA3 models and Ollama form the backbone of our bare-agent workflow, enabling sophisticated language understanding and generation.

3.1.4 | Metaprogramming

Metaprogramming involves writing programs that generate or manipulate other programs, offering flexibility and automation in software development (Štuikys and Damaševičius, 2024). In the context of AI systems, it facilitates structured multi-agent collaboration by embedding workflows, roles, and Standardized Operating Procedures (SOPs) into prompts. This assembly line–like approach reduces hallucinations and conflicting outputs, enhancing coherence and reliability in complex problem solving tasks (Hong et al., 2024).

3.1.5 | Prompt Engineering

A prompt is a brief input that provides instructions and context to guide the output of an LLM (Koul, 2023). However, the effectiveness of a prompt greatly influences the accuracy of the model's response. LLMs are prone to hallucinations, defined as perceiving something unreal as real (Blom, 2023)—they can occur when instructions are ambiguous, informal, or misleading. As Mentzelopoulou (2024) explains, such prompts can introduce uncertainty, reduce precision, and obscure the intent of the user, leading the model to generate inaccurate or irrelevant content. Therefore, crafting clear and structured prompts is essential to minimize hallucinations and improve the reliability of LLM outputs.

3.2 | Web Application

Flask is a lightweight and flexible Python web framework used to develop the backend of the application, handling routing, request processing, and integration with the LLM agent. Its simplicity and modularity make it ideal for rapid development and seamless connection to machine learning workflows (Grinberg, 2018). In this project, Flask manages user inputs, validates game moves, interacts with the LLaMA3 model via Ollama, and returns structured responses for real-time gameplay. Its built-in support for RESTful APIs and templating through Jinja2 allows efficient rendering of game states and dynamic updates within the web interface.

4

Research Methodology

This chapter discusses the stages to be performed by the proponents to achieve the objectives of this study. Figure 4.1 outlines the various stages that were carried out such as conceptualizing game representations, developing a working Damath game engine, developing an LLM Agent with workflows, evaluating the LLM Agent's performance against other AI Players, and developing a Damath application prototype for the game interaction.

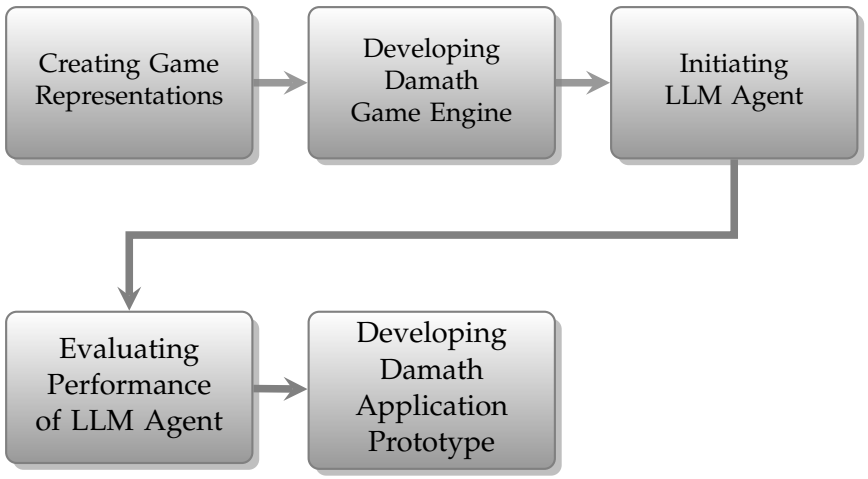


Figure 4.1: Flowchart of the Research Methodologies

4.1 | Damath Game Representations

In order to make the Damath Game easily understandable by the LLM Agent, representations on certain elements of the Damath Game are created. The elements of the game that needed representing were the game board, which is needed as it represents the game's state, and the piece movements (both normal movements and piece capture).

4.1.1 | Board Representation

A Damath board is a two-dimensional (2-D) structure which a square is accessed by coordinates in the format `(row, column)`. The goal is to represent it as a 1-D (one-dimensional) array, so that it can be easily converted into *json* format for abstraction.

As shown in Figure 4.2, the Damath board consists of 64 spaces, half of them being unusable spaces. The board is converted into a 1-D array, starting from 0 up to 63. The white spaces are numbered because these are the spaces that are of concern, which means that the pieces can only travel to these spaces. These numbers indicate the positions of the board.

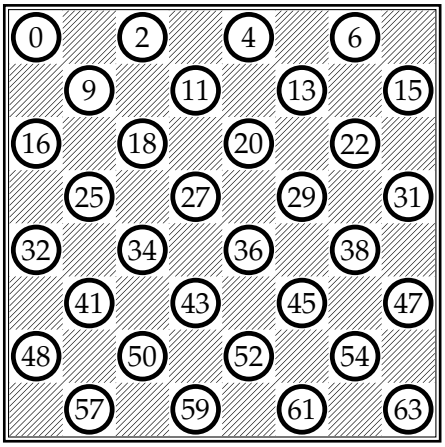


Figure 4.2: One-dimensional Array

From the illustration of the one-dimensional array representation of the board, a mapping of the positions and the operators can be done as depicted in Table 4.1. This will be used by the Agent to determine the operation of a capture move.

Table 4.1: Position-Operator Mapping

Addition	Subtraction
6, 13, 18, 25, 38, 45, 50, 57	4, 15, 16, 27, 36, 47, 48, 59
Multiplication	Division
0, 11, 20, 31, 32, 43, 52, 63	2, 9, 22, 29, 34, 41, 54, 61

In incorporating the board position in Figure 4.2 and the operator mapping in Table 4.1, the *Python* format of the board as shown in Figure 4.3 represents a Damath board as a one-dimensional Python list of 64 elements, corresponding to an 8×8 grid. Each element is either a playable white square or a non-playable black square. Playable squares are represented as two-element lists containing either a `Piece` instance or `None`, and an arithmetic operator (`+`, `-`, `*`, `/`), while non-playable squares are marked as `'X'`. The `Piece` class models a Damath piece with attributes such as color (`'r'` for red, `'b'` for blue), integer value, promotion status (`is_dama`), and additional properties for tracking captures and indexing. This board format enables the agent to efficiently interpret the state, compute valid moves, and apply arithmetic operations during gameplay.

```
1   [
2     [Piece('r', 2, is_dama=False), '*'], 'X', [Piece('r', -5,
      is_dama=False), '/'], 'X', [Piece('r', 8, is_dama=False),
      '-'], 'X', [Piece('r', -11, is_dama=False), '+'], 'X', 'X',
      [Piece('r', -7, is_dama=False), '/'], 'X', [Piece('r', 10,
      is_dama=False), '*'], 'X', [Piece('r', -3, is_dama=False),
      '+'], 'X', [Piece('r', 0, is_dama=False), '-'], [Piece('r',
      4, is_dama=False), '-'], 'X', [Piece('r', -1, is_dama=False),
      '+'], 'X', [Piece('r', 6, is_dama=False), '*'], 'X',
      [Piece('r', -9, is_dama=False), '/'], 'X', 'X', [None, '+'],
      'X', [None, '-'], 'X', [None, '/'], 'X', [None, '*'], [None,
      '*'], 'X', [None, '/'], 'X', [None, '-'], 'X', [None, '+'],
      'X', 'X', [Piece('b', -9, is_dama=False), '/'], 'X',
      [Piece('b', 6, is_dama=False), '*'], 'X', [Piece('b', -1,
      is_dama=False), '+'], 'X', [Piece('b', 4, is_dama=False),
      '-'], [Piece('b', 0, is_dama=False), '-'], 'X', [Piece('b',
      -3, is_dama=False), '+'], 'X', [Piece('b', 10,
      is_dama=False), '*'], 'X', [Piece('b', -7, is_dama=False),
      '/'], 'X', 'X', [Piece('b', -11, is_dama=False), '+'], 'X',
      [Piece('b', 8, is_dama=False), '-'], 'X', [Piece('b', -5,
      is_dama=False), '/'], 'X', [Piece('b', 2, is_dama=False),
      '*']
3   ]
```

Figure 4.3: 1D array format of the Damath board represented as a single Python list.

The one-dimensional board structure is then converted into a *JSON* format for abstraction, depicted in Figure 4.4. Inside it is a board object, indicated by the key `"board"`. Inside the board object is an array representing the valid squares on the board. Each element in the array indicates a value for the position of the square, and the piece on top of it containing information of its color, value, and if it is a Dama piece or not.

```
{
"board": [
    {
        "position": 0,
        "piece": ["color", "value", "isDama"]
    },
    {
        "position": 2,
        "piece": null
    },
    ...
    ]
}
```

Figure 4.4: JSON format of the board.

4.1.2 | Movement Notations

There are two types of movement in Damath. However, they can be represented by using a source position and destination position, as shown on Figure 4.5. Pieces can move to certain free squares on the board, and also jump over other pieces to perform a capture and land on a free square behind the captured piece. For specific mechanics, refer to Section 4.2.1.

```
{
    "source": 2,
    "destination": 9
}
```

Figure 4.5: Agent's output for its piece manipulation in JSON format

4.2 | Damath Game Engine

The game engine is responsible for processing the inputs and outputs of both the Player (Game Client) and the LLM Agent. Processes data such as player moves, agent moves, and game state information between the game client and the LLM agent. As shown in Figure 4.6, the game engine consists of functions responsible for providing valid moves, validating moves, piece-capturing movements, anything that results in creating another game state from a previous one, which are derived from the game mechanics in Section 4.2.1, as well as input/output conversions of player moves, agent moves, and the game state. In addition, the game engine determines whether a game is over, ending the game session.

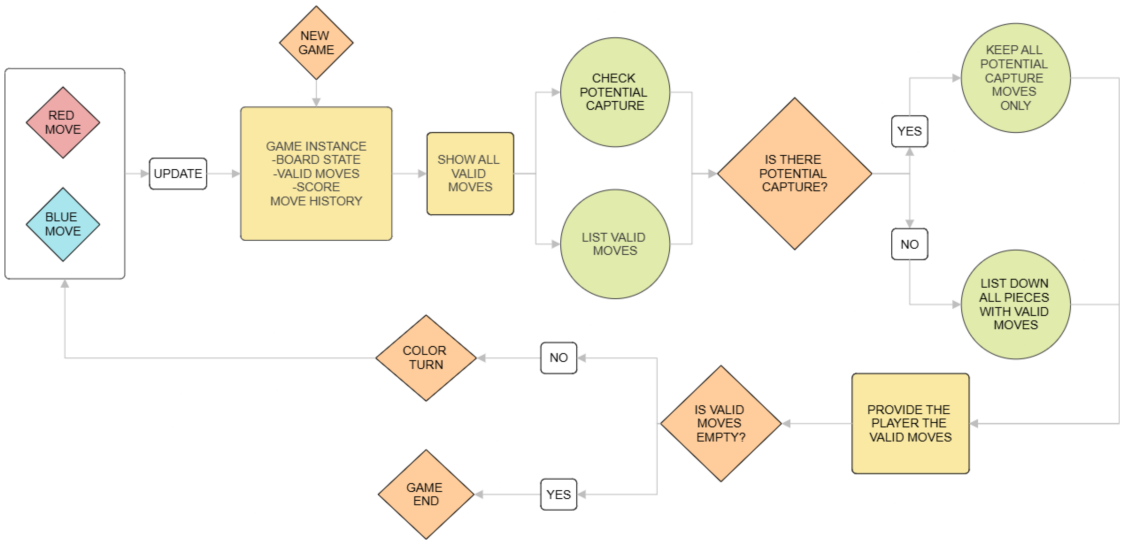


Figure 4.6: Damath Engine Workflow

4.2.1 | Game Mechanics

The mechanics of the Damath game define the rules and behaviors governing how pieces are moved, promoted, captured, and scored throughout the match. Understanding these mechanics is essential to mastering the strategic and mathematical aspects of the game. The sections below break down the movement capabilities of regular and Dama pieces, the conditions and rules for promotion, the mandatory nature of piece captures, and how scores are computed based on in-game interactions.

4.2.1.1 | Piece Movement

A `Piece` can only have a valid move if:

1. it is currently their turn,
2. there are free spaces in front of them,
3. there is an enemy piece behind/in front of them, and there is a free space behind the enemy piece on the same direction (piece capturing)

For Dama pieces, however, they can move on all diagonals no matter how far, until a piece is blocking the way. If that happens, all the spaces behind that blocking piece are not considered valid moves, except if the blocking piece is an enemy piece.

The priority are as follows; its currently their turn, there is an enemy piece in front/behind of them, and there are free spaces in front of them. A piece is not able to be moved backwards, unless that piece is a Dama piece or a piece capturing occurs.

4.2.1.2 | Piece Promotion

A Dama piece is a piece that have reached the farthest row of their enemy's side. When a piece reaches the last row, they undergo promotion into a Dama piece, granting it new abilities. A Dama piece can move on all diagonals, as long as those moves are valid. Even with these unlocked abilities, a Dama piece is not able to jump off of (capture) 2 or more adjacent pieces. When a Dama piece takes an enemy piece, the score will be greater than when a normal piece is the one doing the capturing, which will be discussed in 4.2.1.4.

4.2.1.3 | Piece Capturing

One of the valid moves for a piece is for piece capturing. If any of the pieces has a move that captures an enemy's piece/s, a mandatory capture occurs. When this happens, all other valid moves from other pieces are removed, unless those moves happen to be another mandatory capture. A Dama piece's moves are prioritized if they happen to have a mandatory capture move, resulting in the moves of non-Dama pieces being removed, even if they contain a mandatory capture move.

4.2.1.4 | Scoring

A `Piece` has an attribute `value`, which is used in scoring a piece capture movement. It is calculated by this equation:

$$\text{score} = \{pieceValue\}\{operator\}\{enemyPieceValue\}$$

where *pieceValue* is the value of the piece performing the piece capture, *operator* is the operator on the empty square where the piece will land after capturing, and *enemyPieceValue* is the value of the captured enemy piece. For a Dama piece, it mainly follows the same equation, but the score is doubled after calculation. The doubling of points also works if an ordinary piece captures a Dama piece. However, a Dama piece capturing another Dama piece will quadruple the scores instead.

4.3 | LangChain

The system utilized LangChain's workflow capabilities to construct a code generation pipeline that enables the LLM agent to determine valid moves based on a given Damath board state. This setup prepares the foundational step for issuing a subsequent "best move" instruction. As illustrated in Figure 4.7, the LLM agent follows a structured process that orchestrates prompt templates and tools components to manage stateful interactions.

The models that are used in the LLM Agent come from the LLaMa series (LLaMa 3.1 and LLaMa 3.2). These models were chosen because they were found to be good performance open-source models that are also free to use in the early stages of the study. Moreover, these models conform to the computational and memory constraints of the target hardware platform used to deploy the LLM Agent.

The full implementation and integration details of this workflow will be further explicated in the subsections that follow.

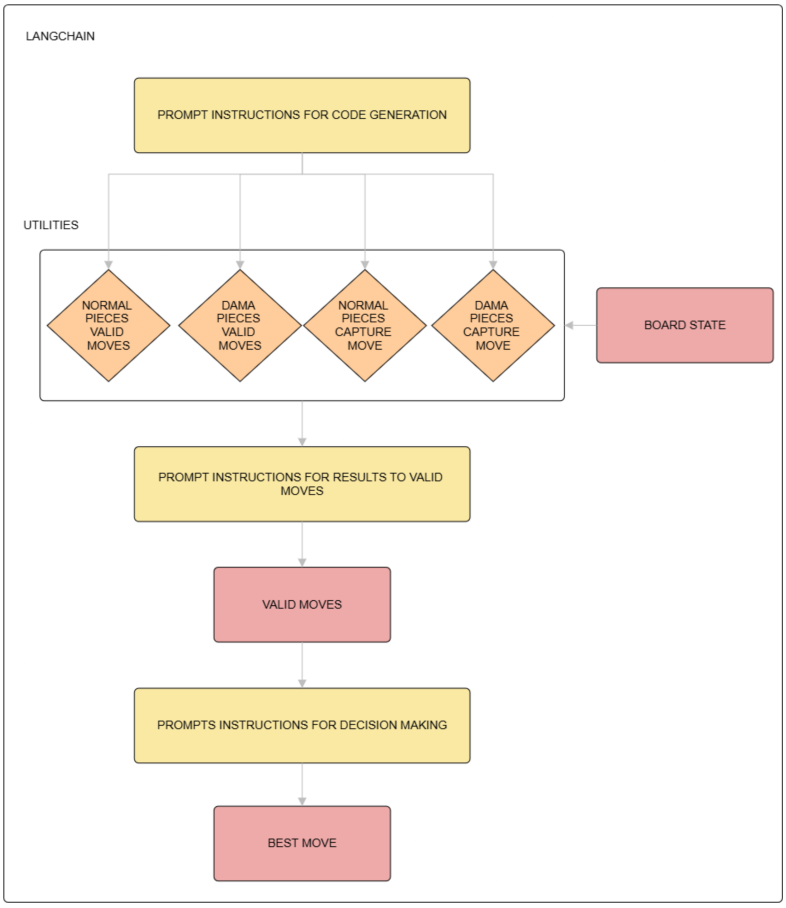
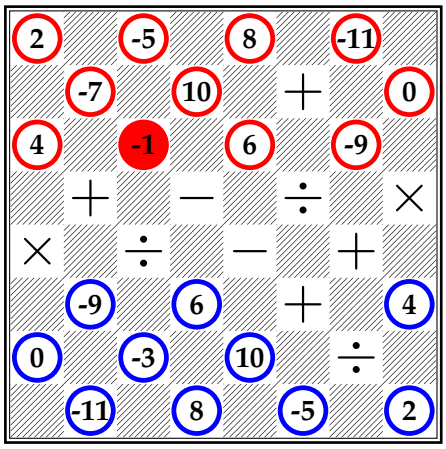


Figure 4.7: Langchain LLM Agent Workflow

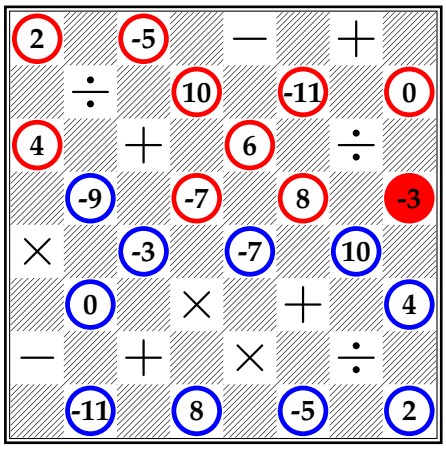
4.3.1 | Code Generation

Utilizing prompt-to-code generation as a utility tool for our agent, a metaprogramming concept that dynamically interprets and executes code based on a given prompt was implemented. This approach enables the agent to generate functional code that computes valid moves from a one-dimensional array representation of the Damath board state (Figure 4.3). This method streamlines the agent's decision-making process and exemplifies the use of generative AI in practical, logic-driven game scenarios. Appendix Figures C.1 and C.2 display the prompt instructions used to generate logic for determining valid moves of normal and dama pieces, respectively. Meanwhile, Appendix Figures C.3 and C.4 showcase the prompts for generating capture logic. These structured prompts serve as the foundation for generating accurate code dynamically, allowing the agent to compute game logic in real time based on the current board state.

An example in Figure 4.8 has an input of board state and its corresponding output from code-generated utilities, it demonstrates the valid normal and dama moves (top) as well as valid normal and dama captures (bottom) for red pieces. These DaMath moves in structured JSON format will then be passed and processed in order to get the final valid moves, which will be discussed in Section 4.3.2.




```
{
  'normal_moves': {4: [13], 6:
  ↪ [13], 16: [25], 18: [25,
  ↪ 27], 20: [27, 29], 22: [29,
  ↪ 31]},
  'dama_moves': {18: [(25, 32),
  ↪ (27, 36, 45, 54), (), ()]}
}
```




```
{
  'normal_moves': {0: [9], 2: [9],
  ↪ 11: [18], 13: [22], 15:
  ↪ [22]},
  'dama_moves': {31: [(), (),
  ↪ (22,), ()]},
  'normal_captures': {27: [45],
  ↪ 29: [43], 31: [45]},
  'dama_captures': {31: [(45, 52),
  ↪ (), (), ()]}
}
```

Figure 4.8: The DaMath Board and its output from code-generated utilities.

4.3.2 | Valid Moves

This stage is critical in the workflow as it determines the legal actions available to the agent, guiding its decision-making process during gameplay. Given the input as seen in Figure 4.8, the prompt instructions in Figure C.5 and C.6 merges and prioritizes these moves and captures based on the following rules:

1. If any captures are available (normal or dama), captures take precedence and moves are disregarded.
2. When both `normal_captures` and `dama_captures` are present, `dama_captures` is prioritized and returns its values in `captures` key as final output.
3. If `dama_captures` are none, and `normal_captures` are present, the `captures` key will contain the values of `normal_captures` as final output.
4. If both `normal_captures` and `dama_captures` are empty, only moves are considered.
5. When merging `normal_moves` and `dama_moves`, if a similar key appears in both, the moves from `dama_moves` are prioritized.
6. The merged moves are combined under a single `moves` key in the final output.

This logic ensures the agent always evaluates rule-compliant moves or captures available on the board.

4.3.3 | Best Move

The best move is selected by invoking the LLM chain with the prompt templates shown in Appendix Figures C.7 (normal-move selection) and C.8 (capture selection). The prompt receives the input of current board state and the list of valid source–destination pairs derived from the final valid moves. This was done to make the input tokens smaller in determining the best move. Finally, it returns a JSON object of the form in Figure 4.5.

4.3.3.1 | Decision Making for Normal Movement

Different instructions are given to the LLM Agent for different cases. A prompt for choosing the best normal movement is used when the source-destination pairs

contain only normal movement. The Agent is given a board state and the list of source-destination pairs. It can use the board state to simulate movements taken from the source-destination pair list. The instruction consists of strategies in choosing a good move. It guides in determining the priorities of what move is ideal, when to use Dama pieces, threat analysis, and more.

4.3.3.2 | Decision Making for Piece Capture Movement

The prompt for deciding which best piece-capture movement is used when the source-destination pair has at least one capture pair. Similar to best move's normal movement, the Agent is given the board state and the list of source-destination capture pairs. However, the prompt advises the agent to choose the capture that yields a higher positive score in the case of multiple capture pairs. The agent is also advised to consider multi-step capture chains that may yield to a higher score.

4.3.4 | Invalid LLM-Generated Valid Moves and Move Choices

A recent study by Sawadogo et al. (2025) was conducted in which they addressed the behaviors of LLMs being non-deterministic. They introduced a Tree-of-Thought (ToT) prompting strategy which mitigates the inherent randomness that is present along its generation process in order to get the ideal near-deterministic outputs. In the context of this study, the non-deterministic behavior of LLMs was dealt with by implementing a model swapping mechanism (LlaMa 3.1 and LLaMa3.2) and a gradual temperature increase mechanism on invalid outputs. Along its Agentic workflow, if the agent does not produce a parsable JSON output, it will rerun the process. For each failed attempt, or if the output remains unparsable, incrementing the model's temperature by 0.04 will increase the possibility of finding more possible parsable outputs. This is also the case for the LLM Agent's decision for the best move. If the agent does not provide a source-destination pair that is not found in the valid moves, then it will rerun the process with increased temperature (similar increment) for determining the best move. In an attempt to determine a best move from a board state, if it fails, a model switch would trigger, and it will continue until attained.

When the LLM Agent has already decided a best move, it will return a JSON object shown in Figure 4.5. Occasionally, this output may not be a valid move in the DaMath Engine. In that case, the chosen source-destination pair by the Agent is popped from the list of source-destination pairs. The process of determining the best move is re-evaluated with the new list of source-destination pairs until the Agent has chosen a valid move.

4.4 | Agent Evaluation

This section discusses the methods used to evaluate the LLM Agent.

4.4.1 | Code Generation Runtime

The code generation that was discussed on Section 4.3.1 will usually take a long time to generate due to the model's limitation. The code generation will be executed five times, with all executions being timed to determine the average runtime.

4.4.2 | Game Playing with Random Choice Generator AI

In order to determine the proficiency of the LLM Agent, it played 50 games against a Random Choice Generator (RCG) AI. The RCG AI gets the valid moves from the engine, and a choice is made by using the built-in module `random`.

4.4.3 | Game Playing with MiniMax AI

To further assess the proficiency of the LLM Agent, the agent played 50 games against a MiniMax opponent. The MiniMax AI was configured with a maximum search depth of 4 plies and employed a heuristic based on the score gap between the opponent and the MiniMax player. The utility function assigned a high weight to this score gap, specifically using a 5x multiplier to emphasize the advantage or disadvantage in scoring. Additionally, the MiniMax AI was optimized to make each move in under 5 seconds, ensuring efficient and timely decision-making throughout the matches.

The MiniMax AI is implemented as a depth-limited alpha-beta pruning minimax algorithm. Since there is a set max depth, there will be leaf nodes on the search space that are non-terminal states. These non-terminal states (ongoing games) will be evaluated using the heuristic shown above. For terminal states (game over), the utility score is used, which is five times the heuristic evaluation score.

4.5 | Damath Prototype

The DaMath prototype established an efficient, streamlined system for automated DaMath gameplay, enabling both LLM Agent and human interactions. Essential strategies and plans such as the development tools, system architecture, technology stack, and various processes were structured.

4.5.1 | System Architecture

Figure 4.9 presents an overview of the system architecture, highlighting three main components: LangChain, the Damath Engine, and the Game Interface.

The player will be a part of the **Game Client**, where the players interact with the game. Additionally, the player will engage in **Game Playing**, which involves several key interactions with the Damath game. These interactions include starting a new game, initiating the first turn, making moves during the game, receiving game state information from the Game Engine, and being informed of the game result once the game is over.

The Game Engine will be the part of the **Game Server** where the Damath game system resides, it acts as the intermediary between the Player (Game Client) and the LLM Agent. It is responsible communicating data such as player move, agent move and game state information from the game client to the LLM Agent, and vice versa. The whole interaction of the game will be stored in a MySQL Database.

The LLM Agent will be a part of the **Game Server**, as it is acts as a counterpart of the player, in **Game Playing** the agent interacts with the Damath game by making moves during the game, receiving game state information from the Game Engine, process via agentic workflow and provide the best move source-destination pair.

4.5.2 | Development Tools and Resources

To implement the Damath game application, the Python Flask framework was selected due to its lightweight architecture, modular design, and strong ecosystem. Flask provides an intuitive routing system, a built-in development server, and seamless integration with extensions for database management, and RESTful APIs (Grinberg, 2018), its minimal core facilitates rapid prototyping and easy maintenance as requirements evolve. The application is developed and tested on the following hardware and software configuration to ensure performance and compatibility:

1. **CPU:** Intel Core i9-12900K (12th Gen, 24 threads) @ 5.10,GHz
2. **Memory:** 32 GB DDR4
3. **GPU:** NVIDIA GeForce RTX 3060 (Lite Hash Rate) and Intel Alder Lake-S GT1 integrated graphics
4. **Operating System:** Ubuntu 24.04.2 LTS (x86_64)

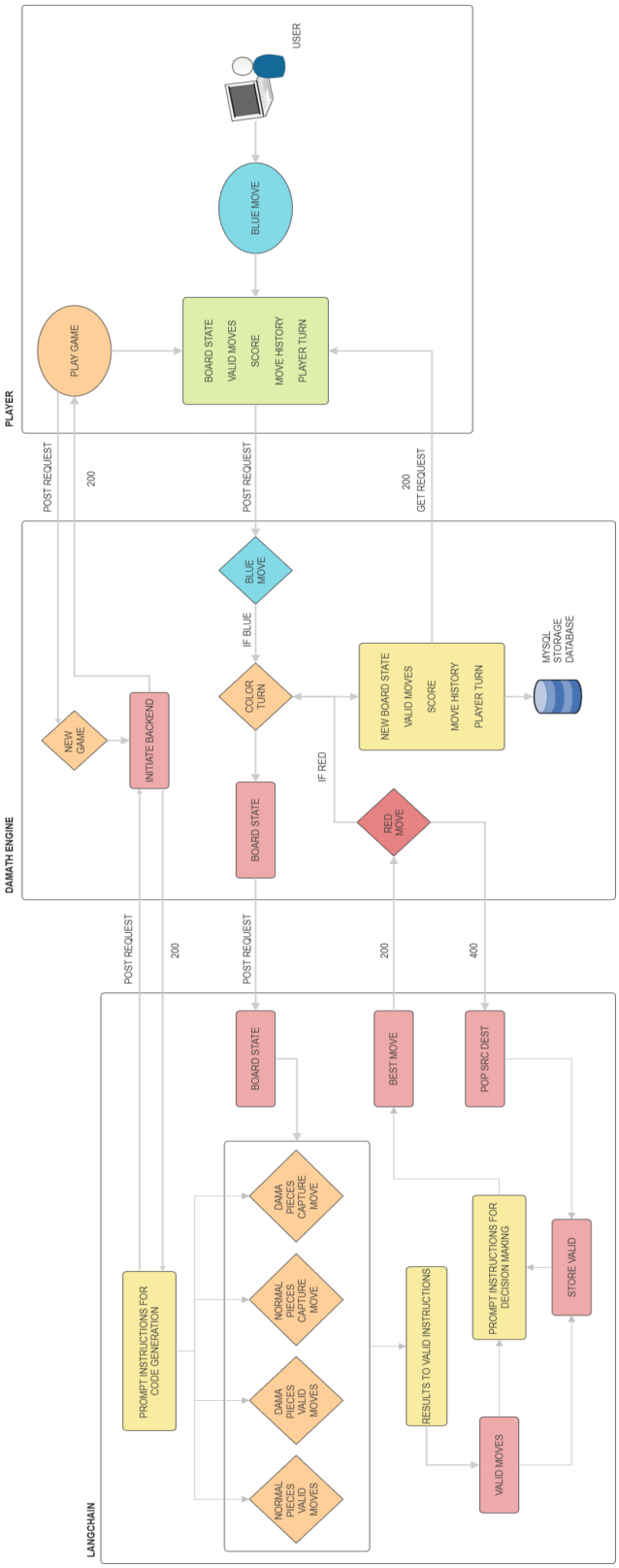


Figure 4.9: Architectural Diagram

5

Design and Implementation Issues

During the course of this study, several critical considerations and challenges emerged that significantly impacted both the research process and its resulting outputs. These issues necessitated iterative revisions to the research methodology and prompted the replacement of certain tools and approaches that were initially selected. This chapter discusses these challenges, particularly in the development of the Retrieval-Augmented Generation (RAG) agent and the fine-tuning process of the model.

5.1 | Development of RAG Agent

One of the designed core components of this study was the development of a RAG agent that could assist in playing DaMath. To provide the agent with the necessary domain knowledge, a curated set of PDF documents was embedded into a vector database. These documents included: containing the history of DaMath, explanations of game representations and mechanics, structured rules for gameplay, annotated examples of board states and possible moves.

The RAG agent utilized these resources to retrieve relevant information and respond contextually to user queries or in-game decisions. The vector store was implemented using PgVector, and the agent was powered by a local instance of LLaMA3 via the Ollama platform.

However, despite the rich retrieved embeddings, the agent's game-playing capabilities were severely limited by the sparse DaMath-specific gameplay data available. We found that the model could not reliably produce valid source and destination moves from a given board state due to insufficient examples illustrating

complete play sequences. An alternative approach considered populating the vector database directly with mappings from board states to all possible moves; but given that the state-space complexity of an DaMath board is on the order of 500 billion positions (Schaeffer et al., 2007), storing and retrieving every state-move pair was infeasible given our computational and storage constraints.

5.2 | Fine-Tuning

Instead of storing the entire vector database for RAG (refer to Section 5.1), another approach was to fine-tune the model to enhance its ability to understand and generate DaMath-specific responses. We constructed a fine-tuning dataset organized into several categories of domain-specific knowledge:

1. Basic DaMath information: Fundamental rules of the game, including piece values, movement directions for red and blue pieces, and the structure of the board.
2. Game mechanics: Detailed capture rules, how arithmetic operations (addition, subtraction, multiplication, and division) influence scoring, the promotion of pieces to "Dama," and overall win conditions.
3. Internal representation: JSON modeling of the board, pieces, and game states. Each square is identified by a position index, and each piece is assigned attributes such as color, value, and Dama status. This structure enabled the model to parse and interact with the game programmatically.
4. Annotated game histories: Historical match sequences illustrating move sequences, strategies, and decision-making patterns to help the agent learn plausible trajectories and effective tactics.

However, a major challenge in our fine-tuning effort was handling and debugging model hallucinations. For example, when querying a board state, the fine-tuned model returned results in the correct format, but the source and destination indices were inaccurate and did not correspond to valid moves in the Damath game engine.

5.3 | Model Selection

During the early stages of LLM Agent creation, a different model was used to determine its valid moves and to choose the best move. A quantized model of LLaMa 3.2 was used due to hardware limitations, sacrificing the quality of the model. In the completion of the LLM Agent, another device was used, which is more capable than the previous device. The new device was capable of running models that did not require quantization (LLaMa 3.1 and 3.2).

5.4 | Prompt Instruction Development

Instructions for filtering the results from the code generation process to a finalized list of valid moves and determining the best move mentioned in Sections 4.3.2 and 4.3.3 were initially designed as two large prompts. The creation of these prompts were made with the assistance of helpful agents such as ChatGPT and Deepseek.

However, this approach resulted in the LLM struggling to follow the instructions to perform its task, especially on very simple tasks such as determining whether the results contain only move choices or if there are capture choices as well. Additionally, the instructions that ChatGPT and Deepseek provided turned out to be ineffective against the LLM Agent.

With that in mind, the large prompts were then divided into subtasks, creating prompt instructions for the final valid moves to determine whether the results contain only move choices or if there are capture choices as well, and then deciding which parts of the results to keep, following the ruleset in Section 4.3.2. For the best move, the instructions were split into two for move choices and capture choices. The original prompts that were generated by ChatGPT and Deepseek served solely as a basis for crafting the new prompt instructions. However, these revised instructions were then substantially simplified to ensure that the LLM agent could accurately understand and perform the tasks.

This newly derived approach worked better than the initial approach, but not without flaws. There are still cases where the LLM Agent generates unparsable or purely incorrect output. In response to this issue, a workaround was developed to ensure that the LLM Agent will eventually provide an acceptable response. This workaround method is discussed in Section 4.3.4.

5.5 | Summary

Given that DaMath moves are fully deterministic, approaches based on probabilistic language model outputs—such as RAG retrieval and fine-tuning—are prone to hallucinations and inconsistencies when tasked with generating precise move indices. To ensure reliable, deterministic move selection, we therefore pivot to an agentic workflow that leverages our preferred model's strengths in code generation prompting game logic as executable code rather than relying solely on natural language outputs.

6

Results & Discussion

This chapter consists of all evaluations of the LLM Agent, as well as discussions on the results obtained. Additionally, the web application for the Damath Game Prototype is also shown here.

6.1 | LLM Agent

The assessment focuses on three main areas: the agent's average code generation runtime, its game-playing capability when paired with the baseline Random Choice Generator (RCG) AI, and its performance against a MiniMax AI algorithm. Each subsection below outlines the methodology, execution, and results of the corresponding evaluation scenario.

6.1.1 | Code Generation Runtime

Before the LLM Agent is able to play, it has to generate codes to determine the different types of valid moves when given a board state. The code generation process was executed 5 times, with each instance having varying results. Refer to Table 6.1 to see the values on each runtime.

As shown in Table 6.1, the average runtime for the code generation process is 24 minutes and 11.51 seconds. Compared to the other instances, the fastest runtime of all instances is the 3rd instance with a runtime of 3 minutes and 2,38 seconds to complete the code generation process. The second fastest is the 4th instance, which took only 13 minutes and 3.60 seconds. All other instances apart from these two were in between 20 to 50 minutes.

Table 6.1: Code Generation Runtime in 5 Instances

Instance	Time
1	32m 16.88s
2	47m 4.917s
3	3m 2.38s
4	25m 29.75s
5	13m 3.60s
Average Runtime	24m 11.51s

The variability in execution time appears to be largely stochastic: in some instances would mostly take almost an hour to execute. Nevertheless, it will complete the code generation process, allowing the LLM Agent to play the game.

6.1.2 | Game Playing with Random Choice Generator

The Random Choice Generator (RCG) AI is the simplest possible opponent strategy: whenever it's their turn, the RCG AI first check all its valid pieces from the board that populates the valid moves with every legal source-to-destination pairing for RCG pieces. It then flattens that mapping into a single list of source-destination pairs, selects one of these valid pairs uniformly at random via `random.choice(...)` and executes its move to the game. Because it makes no tactical assessment, no look-ahead, no value computation, and no positional reasoning—theRCG AI serves as a baseline or "dummy" opponent, useful for testing the engine's mechanics or benchmarking more sophisticated agents.

The Table 6.2 shows each row representing the outcome of a single game, identified by a unique Game ID. The Move Length column indicates the total number of moves made during that game, serving as a measure of its duration or complexity. The LLM agent score and the RCG score denote the final points accumulated by both players, respectively. The Winner column shows which player emerged victorious based on the higher score. Lastly, the Score Difference reflects the margin by which the winner outscored the opponent, highlighting the competitiveness or dominance observed in each match.

Table 6.2: RCG AI Game Results across 50 Games.

Game ID	Move Length	LLM Agent Score	RCG AI Score	Winner	Score Diff.
1	57	78	-17	RCG AI	95
2	49	-3	40	LLM Agent	-43
3	48	-17	39	LLM Agent	-56
4	43	-27	-42	RCG AI	15
5	59	92	-67	RCG AI	159
6	59	14	-40	RCG AI	54
7	56	-117	-85	LLM Agent	-32
8	67	-3	-25	RCG AI	22
9	115	100	-15	RCG AI	115
10	49	199	-60	RCG AI	259
11	39	93	-106	RCG AI	199
12	44	95	-75	RCG AI	170
13	66	-191	-43	LLM Agent	-148
14	59	-29	13	LLM Agent	-42
15	60	113	-52	RCG AI	165
16	42	117	42	RCG AI	75
17	44	-107	-4	LLM Agent	-103
18	46	-10	-16	RCG AI	6
19	54	-61	104	LLM Agent	-165
20	44	-81	-5	LLM Agent	-76
21	43	-1	17	LLM Agent	-18
22	54	-32	-11	LLM Agent	-21
23	42	222	8	RCG AI	214
24	72	53	-40	RCG AI	93
25	50	22	-11	RCG AI	33
26	49	-13	-146	RCG AI	133
27	62	-101	-21	LLM Agent	-80
28	58	-85	-61	LLM Agent	-24
29	37	-113	-80	LLM Agent	-33
30	46	155	3	RCG AI	152
31	43	15	-42	RCG AI	57
32	39	-138	45	LLM Agent	-183
33	47	-19	11	LLM Agent	-30
34	50	5	-7	RCG AI	12
35	102	105	-11	RCG AI	116
36	50	1	-21	RCG AI	22
37	53	-36	-79	RCG AI	43
38	56	-22	-42	RCG AI	20
39	57	46	-112	RCG AI	158
40	63	158	82	RCG AI	76
41	36	-89	36	LLM Agent	-125
42	50	-83	8	LLM Agent	-91
43	64	133	51	RCG AI	82
44	47	-8	68	LLM Agent	-76
45	56	1	-11	RCG AI	12
46	63	8	-136	RCG AI	144
47	56	74	12	RCG AI	62
48	72	-23	3	LLM Agent	-26
49	62	49	-43	RCG AI	92
50	48	12	-27	RCG AI	39

The referenced Figure 6.1 above shows the move history length for each of the 50 games played between the LLM Agent and the RCG AI. The number of moves per game varies significantly, ranging from the lowest being 37 to as many as 115. Despite this variability, no consistent trend is evident in terms of which side benefits from longer or shorter games. Some of the LLM Agent's largest wins occurred in both shorter and longer games (e.g., Games 8 and 10 in Table 6.2), while the RCG AI's occasional victories are similarly scattered across the move-length range. This supports the earlier correlation analysis ($r \approx 0.19$), suggesting that move length is only weakly associated with the final score difference. Overall, the chart emphasizes the variability in game duration when moves are randomized and demonstrates the consistent effectiveness of the LLM Agent across games of different lengths.

The visualization of the cumulative score difference of LLM Agent and RCG AI in Figure 6.2 shows over the course of 50 games. Most trajectories in the first 10-15 moves remain close to zero, reflecting the RCG AI's occasional fortuitous gains. Shortly thereafter, however, nearly all curves ascend steadily into positive territory, indicating the LLM Agent establishing and compounding an early advantage on its scores. By the move ranges 30 to 40, the spread has widened considerably: many games show the LLM Agent leading by 100 to 200 points, with some standout performances reaching over +250, representing its most effective plays. Only a handful of trajectories dip below zero in mid-game, and even these recover to finish with a positive margin. This pattern underscores that, while the RCG AI can sporadically contest the opening, its lack of strategic evaluation allows the LLM Agent's evaluations to translate small early leads into decisive, runaway victories as the game progresses.

The box plot in Figure 6.3 presents the final scores for the LLM Agent versus the RCG AI over 50 games. The LLM Agent's scores span from –191 to +222 with a mean of +11.02, indicating it more often achieves positive outcomes. In contrast, the RCG AI's scores range from –146 to +199 with a mean of –19.42, showing that RCG AI typically yields negative scores against the LLM Agent. On average, the LLM Agent finished with a positive score of 11.02 points per game, whereas the RCG AI averaged -19.42, yielding an average score difference of $+30.44$ in favor of the LLM Agent. The LLM Agent distribution is shifted well above zero, while the RCG AI distribution lies predominantly below, depicting that the LLM Agent significantly outperforms the RCG AI on average. Both distributions shows outliers—particularly on the positive end for the LLM Agent—but the central tendency and interquartile range (IQR) make clear the

LLM Agent's superior and more consistent scoring compared to RCG AI.

As evident from the Figure 6.4, the LLM Agent consistently outperformed the RCG AI in over 50 games, the LLM Agent secured 31 victories (62%) compared to the RCG AI's 19 (38%), with no ties. This clear majority for the LLM Agent underscores its ability to consistently convert strategic evaluations into wins against the RCG AI. However, the RCG AI's 19 wins reveal that, on occasion, chance can exploit moments where the Agent's move selection is suboptimal, suggesting potential areas for strengthening the Agent's decision-making in less clear-cut positions.

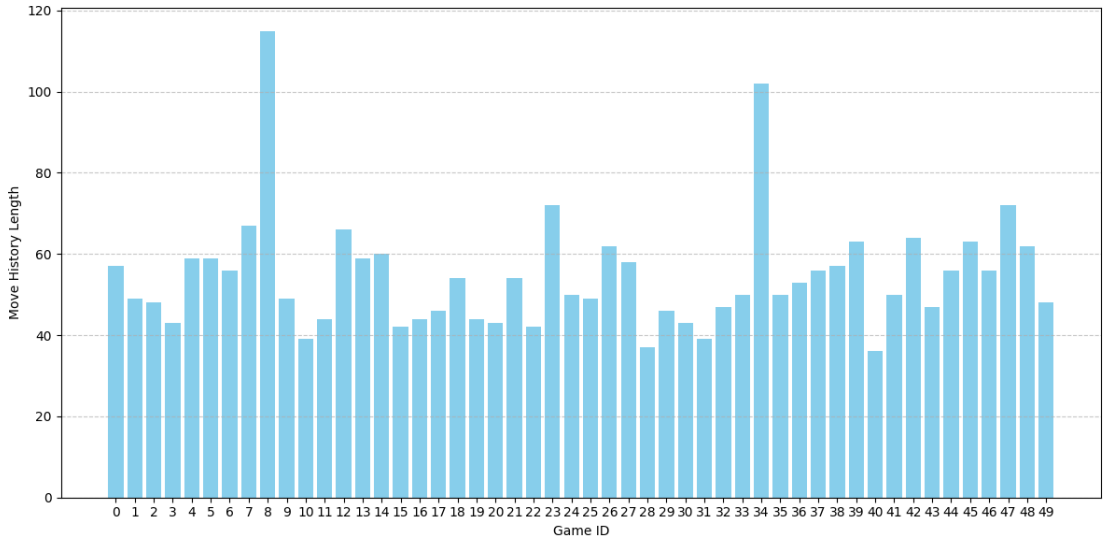


Figure 6.1: RCG Move History Length for 50 Games

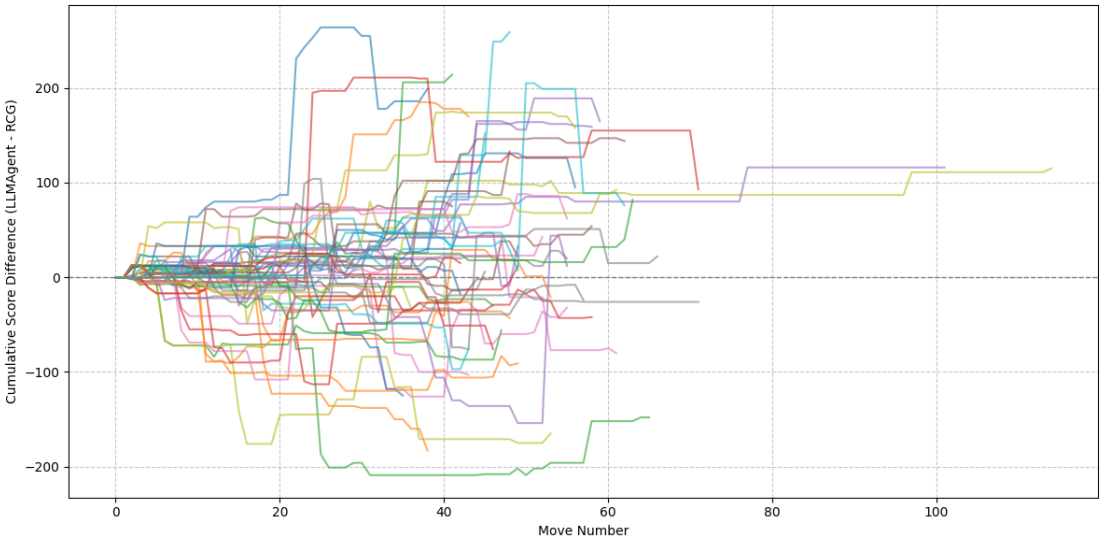


Figure 6.2: RCG Cumulative Score Difference Across Moves (50 Games)

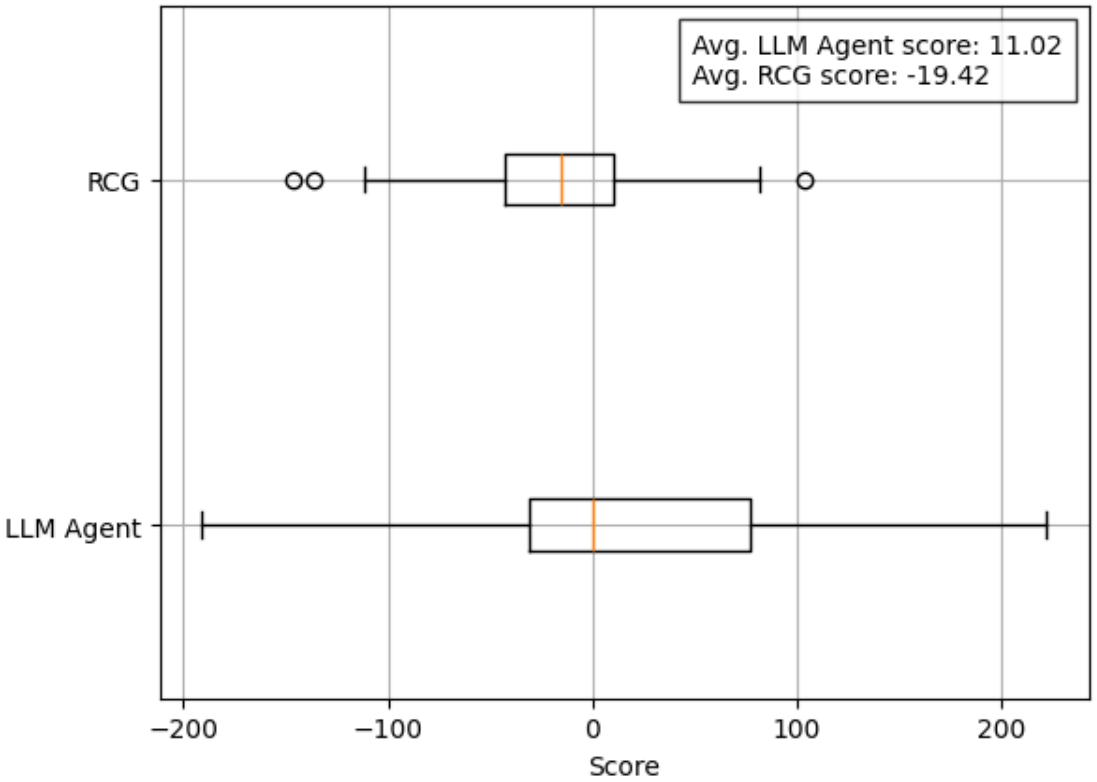


Figure 6.3: RCG Boxplot for Comparing LLM Agent and RCG AI Scores

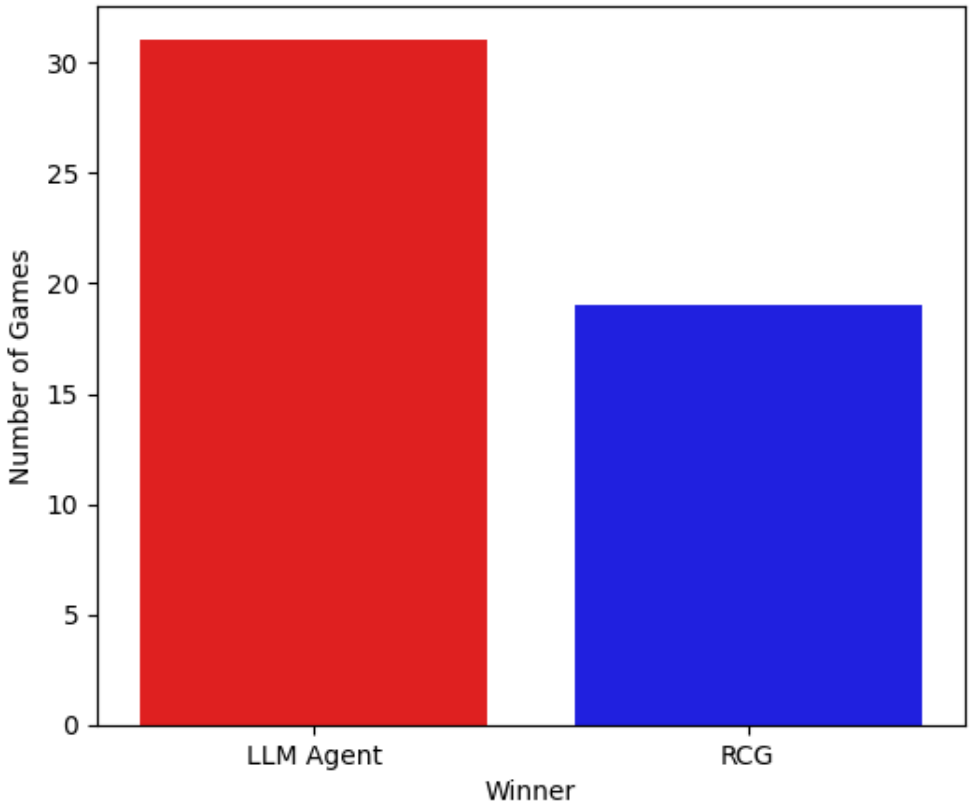


Figure 6.4: RCG Game Outcomes

6.1.3 | Game Playing with MiniMax

The MiniMax AI takes in a game state, where it uses the board state to check all valid moves on the MiniMax AI's pieces. It then performs all of the valid moves using the previous game state, to create a new game state where it identifies all valid moves for the LLM Agent's pieces. It reiterates this process until the max depth is achieved, or a terminal state is achieved. When the max depth is achieved, that game state is a non-terminal state. The MiniMax AI evaluates this state using a heuristic evaluation function where it is assessed by determining the score difference of the two players. However, terminal states uses a utility function which is similar to the heuristic evaluation function, but is multiplied by 5 times, indicating that the terminal states have more importance than non-terminal states. The MiniMax AI will then choose the best move to be passed on to the engine. This AI is expected to be the best-performing player

compared to the Random Choice Generator (RCG) AI and the LLM Agent.

Similar to Table 6.2, the Table 6.3 also shows each row representing the outcome of a game, identified by a unique Game ID, Move Length, LLM Agent Score, MiniMax AI Score, Winner and Lastly, the Score Difference.

Figure 6.5 presents a bar chart showing the move history length for each of the 50 games played between the LLM Agent and the MiniMax AI. The number of moves per game varies slightly, but no consistent trend is evident in terms of which side benefits from longer or shorter games. The negative correlation value suggests a slight association between game length and the extent of the LLM Agent's underperformance, given that most final score differences are negative. The spread of outcomes across the 50-game trial is quite large ($\sigma \approx 76.26$), ranging from the LLM Agent's most dominant win—a 38-point margin in Games 10, 16, 20, and 22—to its worst loss, a staggering 322-point deficit in Game 5. Although game lengths varied from as few as 32 moves to as many as 56, the correlation between move length and final score margin is weak ($r \approx -0.317$), indicating that performance is not strongly tied to the game's duration. Overall, this 50-game trial demonstrates that while the games tend to be relatively fast and close to the average in length, the LLM Agent decisively underperforms the MiniMax AI, exhibiting a high degree of score volatility.

As depicted in Figure 6.6, in the opening 5–10 moves the cumulative score difference between the two sides remains tightly clustered around zero, indicating a well-balanced early game. After this point, variance steadily increases: some trajectories drift decisively downward (favouring MiniMax AI) and a few remain nearly flat, ending in very close contests. By the mid-game (around moves 20–30), significant separation emerges, with many games exhibiting runaway leads of ±50–100 points or more. In the late game (moves 40+), the majority of matches have already swung decisively one way, with final score differences ranging in the low hundreds. This growing spread over time suggests that, while MiniMax play keeps the matchup balanced at first, small early advantages tend to compound, leading to increasingly lopsided outcomes as the game progresses.

Table 6.3: MiniMax Game Results across 50 Games.

Game ID	Move Length	LLM Agent Score	MiniMax AI Score	Winner	Score Diff.
1	44	-129	20	MiniMax AI	-149
2	42	-106	27	MiniMax AI	-133
3	47	-15	7	MiniMax AI	-22
4	38	-83	14	MiniMax AI	-97
5	38	4	0	LLM Agent	4
6	46	-40	282	MiniMax AI	-322
7	44	0	0	DRAW	0
8	38	4	0	LLM Agent	4
9	43	-42	33	MiniMax AI	-75
10	50	-53	44	MiniMax AI	-97
11	36	38	0	LLM Agent	38
12	42	-106	27	MiniMax AI	-133
13	34	3	19	MiniMax AI	-16
14	38	4	0	LLM Agent	4
15	43	-122	19	MiniMax AI	-141
16	35	-178	4	MiniMax AI	-182
17	36	38	0	LLM Agent	38
18	43	-122	19	MiniMax AI	-141
19	55	-104	36	MiniMax AI	-140
20	46	-4	173	MiniMax AI	-177
21	36	38	0	LLM Agent	38
22	40	-119	85	MiniMax AI	-204
23	36	38	0	LLM Agent	38
24	42	-106	27	MiniMax AI	-133
25	50	-85	21	MiniMax AI	-106
26	33	-137	19	MiniMax AI	-156
27	46	-15	3	MiniMax AI	-18
28	43	-122	19	MiniMax AI	-141
29	42	-63	0	MiniMax AI	-63
30	41	-54	27	MiniMax AI	-81
31	47	-36	40	MiniMax AI	-76
32	42	-105	106	MiniMax AI	-211
33	36	-31	12	MiniMax AI	-43
34	44	28	39	MiniMax AI	-11
35	32	-5	62	MiniMax AI	-67
36	46	-16	11	MiniMax AI	-27
37	41	-43	0	MiniMax AI	-43
38	43	-122	19	MiniMax AI	-141
39	42	9	25	MiniMax AI	-16
40	40	-6	10	MiniMax AI	-16
41	45	-3	11	MiniMax AI	-14
42	47	-5	50	MiniMax AI	-55
43	42	-51	12	MiniMax AI	-63
44	56	-47	117	MiniMax AI	-164
45	41	-71	18	MiniMax AI	-89
46	45	-23	11	MiniMax AI	-34
47	41	-43	0	MiniMax AI	-43
48	43	-122	19	MiniMax AI	-141
49	47	-106	19	MiniMax AI	-125
50	42	-40	28	MiniMax AI	-68

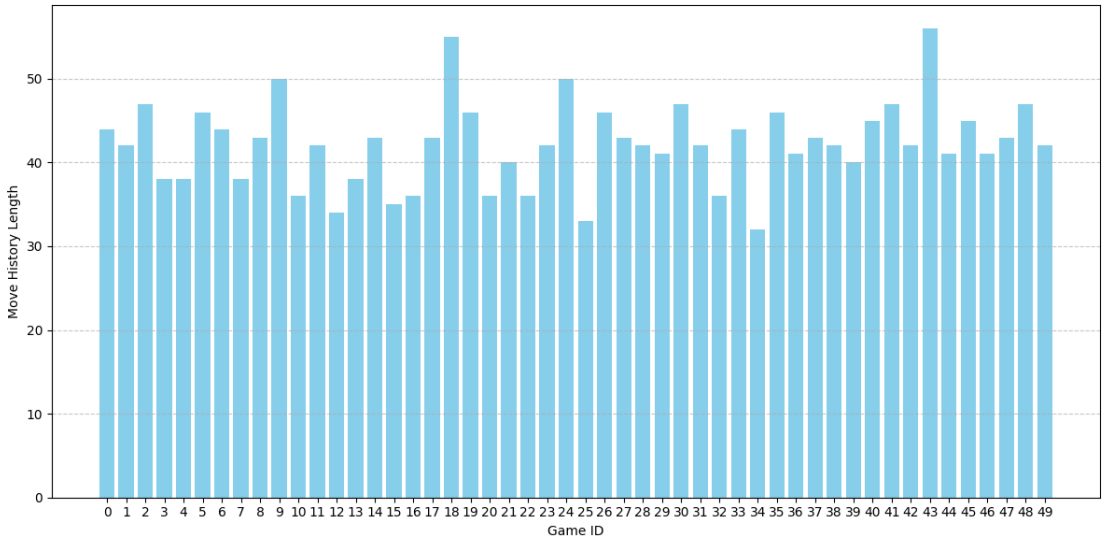


Figure 6.5: MiniMax Move History Length for 50 Games

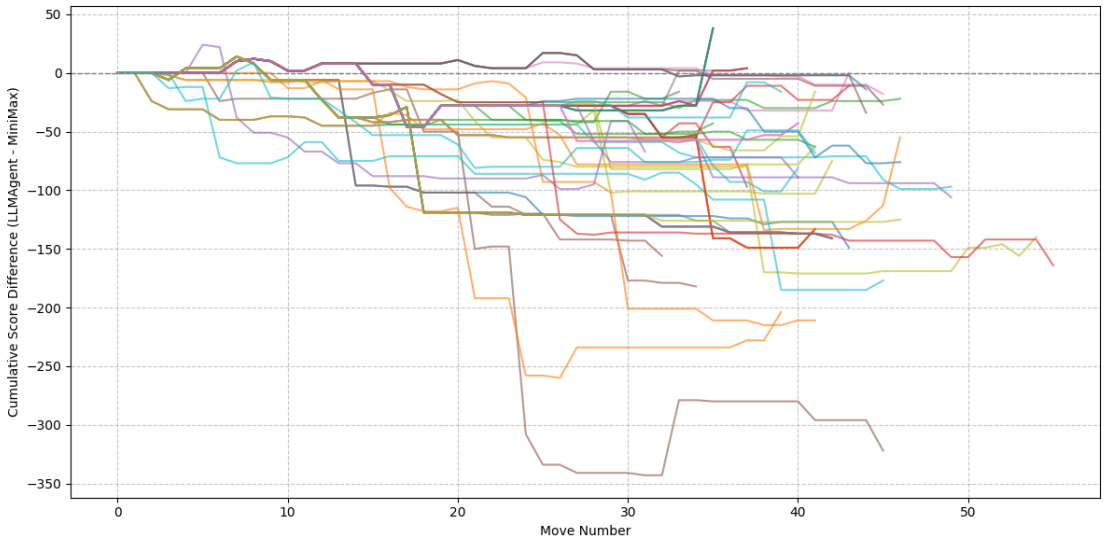


Figure 6.6: MiniMax Cumulative Score Difference Across Moves (50 Games)

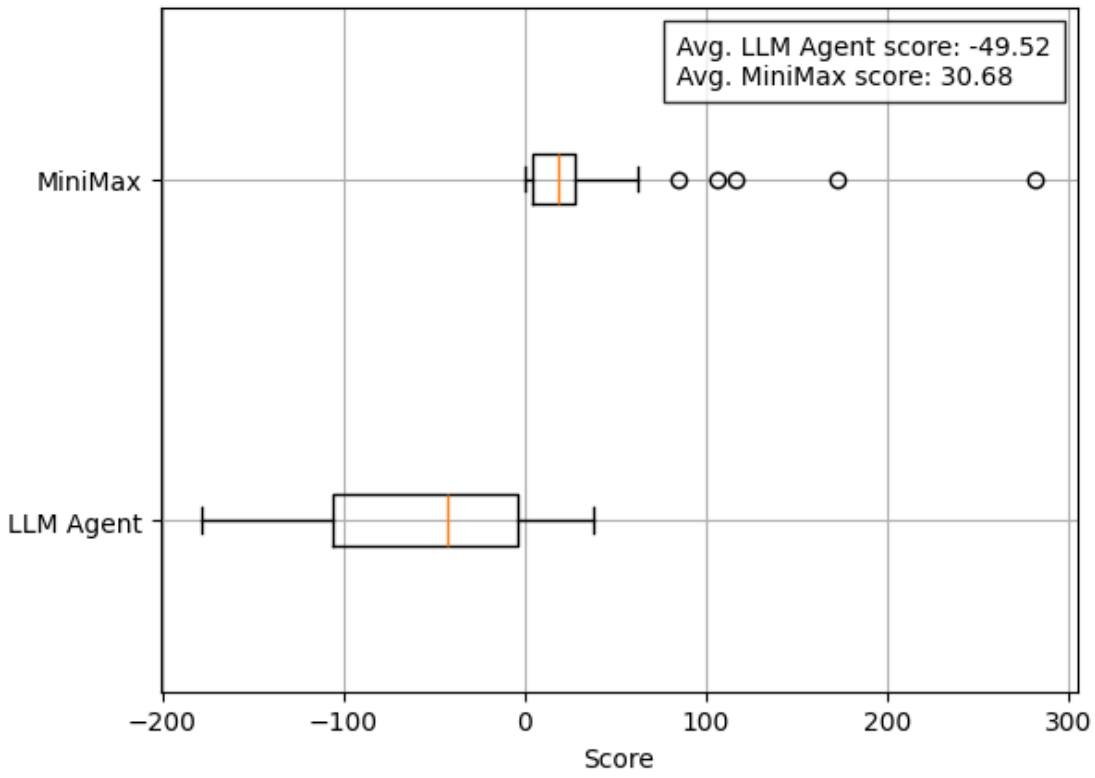


Figure 6.7: MiniMax Boxplot for Comparing LLM Agent and MiniMax AI Scores

The box plot of final scores for the LLM Agent and the MiniMax opponent over 50 games (Figure 6.7). The LLM Agent's scores range from –178 to +38, with a median of –42.5 and an interquartile range (IQR) of 101.5 points; its mean is –49.52. In contrast, the MiniMax opponent's scores span 0 to 282, with a median of 19.0 and a tighter IQR of 23.0 points; its mean is 30.68, yielding an average score difference of -80.2 in favor of the MiniMax AI. These distributions show that MiniMax not only outperforms on average—achieving consistently positive scores—but also exhibits less variability, whereas the LLM Agent's results are both lower and far more erratic, indicating unstable evaluation and move selection in many positions.

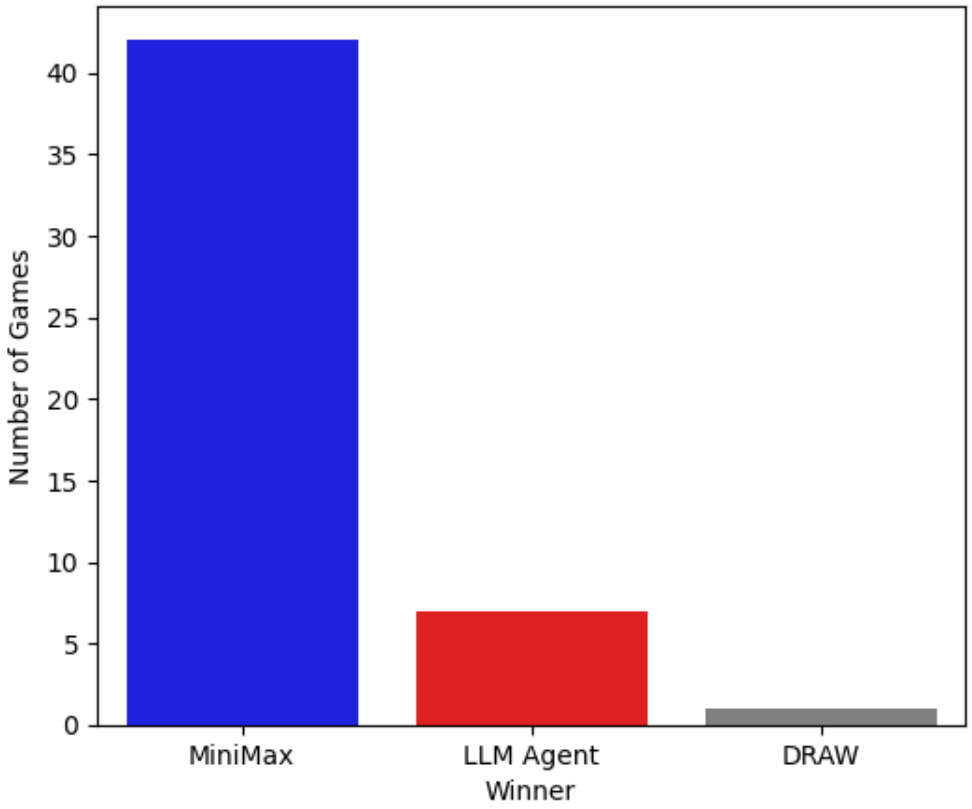


Figure 6.8: MiniMax Game Outcomes

In Figure 6.8, it shows that the LLM Agent underperforms when it is playing against the MiniMax AI over 50 games, the LLM Agent only secured 7 victories (14%) compared to the MiniMax AI's 42 (84%), with only 1 draw (2%). This pronounced skew toward MiniMax victories highlights that, under the current setup, the classical search-based baseline significantly outperforms the LLM Agent. Moreover, the narrow win margins in the few LLM successes versus the large deficits in its losses point to specific weaknesses in the Agent's evaluation and move-selection process that warrant further tuning.

6.2 | Hallucinations

LLMs are known for their limitation in which they tend to hallucinate, wherein they generate information that may be inaccurate or fabricated. The LLM agent is also vulnerable to this phenomenon. This section will cover the LLM's reasoning during each of its move choices, which will be used to determine the percentage of hallucination by the LLM Agent.

Table 6.4: Game History with LLM Agent's Reasoning

Move	Color	Valid Moves	Choice	Reasoning
1	b	[[41, 32], [41, 34], :.	[47, 38, 0]	
2	r	[[16, 25], [18, 25], ...	[16, 25, 0]	advancement
3	b	[[38, 29], [38, 31], ...	[54, 47, 0]	
4	r	[[9, 16], [18, 27], ...	[9, 16, 0]	Advancing toward the opponent's back rank to gain positional strength
5	b	[[38, 29], [38, 31], ...	[61, 54, 0]	
6	r	[[0, 9], [2, 9], ...	[0, 9, 0]	Advancing toward the opponent's back rank (especially pieces near promotion at index 63)
7	b	[[38, 29], [38, 31], ...	[38, 29, 0]	
8	r	[[20, 38, 10], [22, 36, -13]]	[20, 38, 10]	Capture with highest positive score of 10 points. Dama piece at position 20 can jump to position 38 through a +18 offset capture.
9	b	[[45, 31], [47, 29]]	[47, 29, -2]	
10	r	[[22, 36, -2]]	[22, 36, -2]	Capturing a blue piece with value -3 at position 36 with a score of -2.
11	b	[[43, 29], [45, 27]]	[45, 27, 8]	
12	r	[[18, 36, 0]]	[18, 36, 0]	"Capture score: 0, chosen due to single capture opportunity at offset +18"
13	b	[[43, 29]]	[43, 29, -6]	
14	r	[[9, 18], [11, 18], ...	[11, 18, 0]	Advancing toward the opponent's back rank
15	b	[[29, 20], [29, 22], ...	[29, 22, 0]	
16	r	[[13, 31, -18], [15, 29, 0]]	[15, 29, 0]	"Captured a blue piece with a score of 0 at position [29,/]."
17	b	[[41, 32], [41, 34], ...	[41, 34, 0]	
18	r	[[25, 43, -36]]	[25, 43, -36]	Capture with highest positive score (-36) from square 25 to 43
19	r	[[43, 61, 0]]	[43, 61, 0]	Capture at offset -14 with score 0

20	r	[[61, 47, 18]]	[61, 47, 18]	"Capture score of 18 with a Dama piece, moving from position 61 to 47"
21	b	[[48, 41], [50, 41], ...	[50, 41, 0]	
22	r	[[2, 11], [4, 11], ...	[4, 11, 0]	Advancing toward the opponent's back rank
23	b	[[41, 32], [41, 34], ...	[59, 50, 0]	
24	r	[[6, 15], [11, 20], ...	[11, 20, 0]	Advancing toward the opponent's back rank (especially pieces near promotion at index 63)
25	b	[[41, 32], [41, 34], ...	[50, 43, 0]	
26	r	[[2, 11], [6, 15], ...	[2, 11, 0]	Advancing toward the opponent's back rank and protecting high-value pieces
27	b	[[41, 32], [41, 34], ...	[43, 36, 0]	
28	r	[[29, 43, 0]]	[29, 43, 0]	Capturing the blue piece at position 43 with a score of 0
29	b	[[41, 32], [41, 34], ...	[57, 50, 0]	
30	r	[[43, 57, -11]]	[43, 57, -11]	Capture score: -11. Moved from Dama piece at position 43 to capture blue piece at position 57.
31	b	[[41, 32], [41, 34], ...	[41, 32, 0]	
32	r	[[6, 15], [13, 22], ...	[13, 22, 0]	Advancing toward the opponent's back rank
33	b	[[32, 25], [48, 41], ...	[32, 25, 0]	
34	r	[[16, 34, 2], [18, 32, -30]]	[16, 34, 2]	Capture score: 2. Rationale: Single capture with the highest positive score at offset +14.
35	b	[[48, 41], [63, 54]]	[48, 41, 0]	
36	r	[[34, 48, -7]]	[34, 48, -7]	Capture score of -7, chosen due to single capture opportunity"
37	b	[[63, 54]]	[63, 54, 0]	
38	r	[[47, 61, 4]]	[47, 61, 4]	Capture with a positive score of 4 points. Dama piece at position 47 can jump to position 61 through a +14 offset capture.

Table 6.4 shows information about one game instance of the LLM Agent playing with the MiniMax AI. The valid moves, as well as their choices, and the reasoning for the LLM Agent are included. In order to determine the percentage of hallucinated reasons, the valid moves and the move choice will be considered.

In reviewing the LLM Agent's move-by-move explanations, we find that out of the 20 turns where reasoning was explicitly provided, four contain clear inaccuracies—yielding a hallucination rate of 20 percent. Specifically, on Move 8 the agent treats the source as a Dama even if it is a normal piece, however it is a good choice in comparison between scores "–13" and "+10,"; on Move 10 it claims to capture a blue piece value of –3, when in fact the piece's true value is –7; on Move 18 it asserts that –36 is the highest positive score available from square 25 to 43, despite –36 being the only legal (and mandatory) capture but clearly negative; and on Move 30 it again misidentifies the piece as a Dama prior to the capture, when promotion occurs only afterward. These four errors illustrate that while the agent often grasps the broad tactical framework, its numerical and status-based justifications can drift from the actual game state.

Moreover, the agent's multi-capture logic is consistently good: when presented with more than one choice, it always selects the capture with the highest score point gain. For example, in Move 26 it was annotated as "advancing toward the opponent's back rank and protecting high-value pieces," the actual play prioritized safeguarding high-valued key piece—indeed a defensively good choice rather than mere forward advancement. Overall, the agent excels at evaluating capture sequences to maximize score, yet its broader piece-movement decision remains questionable—defaulting to "advancing toward the opponent's back rank" or similar, indicating a gap in strategic nuance in game playing.

6.3 | Web Application

This section tackles the how-tos in interacting with the graphical user interface of the DaMath application.

6.3.1 | User Interface

The DaMath web application offers an intuitive graphical interface designed to help users navigate and play the game with ease. The interface consists of the main game board, control panel, player scorecards, and interaction buttons such as move, reset, and hide/show pieces. The design and layout of the user interface were inspired by the modern checkers GUI from the open-source project by `miskibin` on GitHub (Skibiński, 2023).

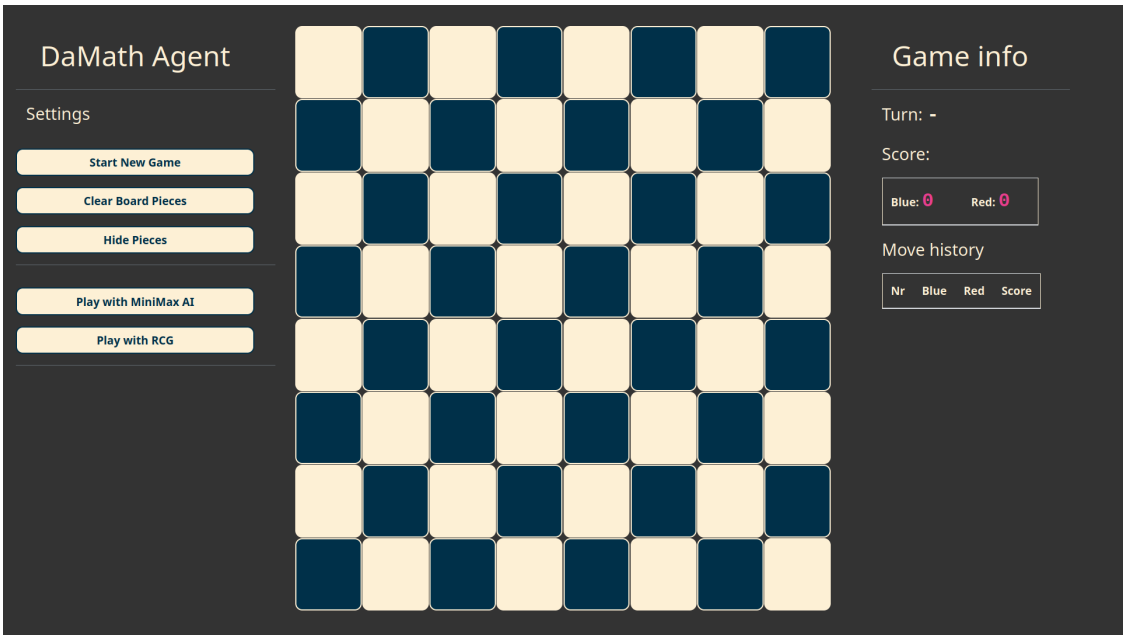


Figure 6.9: Main User Interface of the DaMath Application showing the full layout including the board and controls.

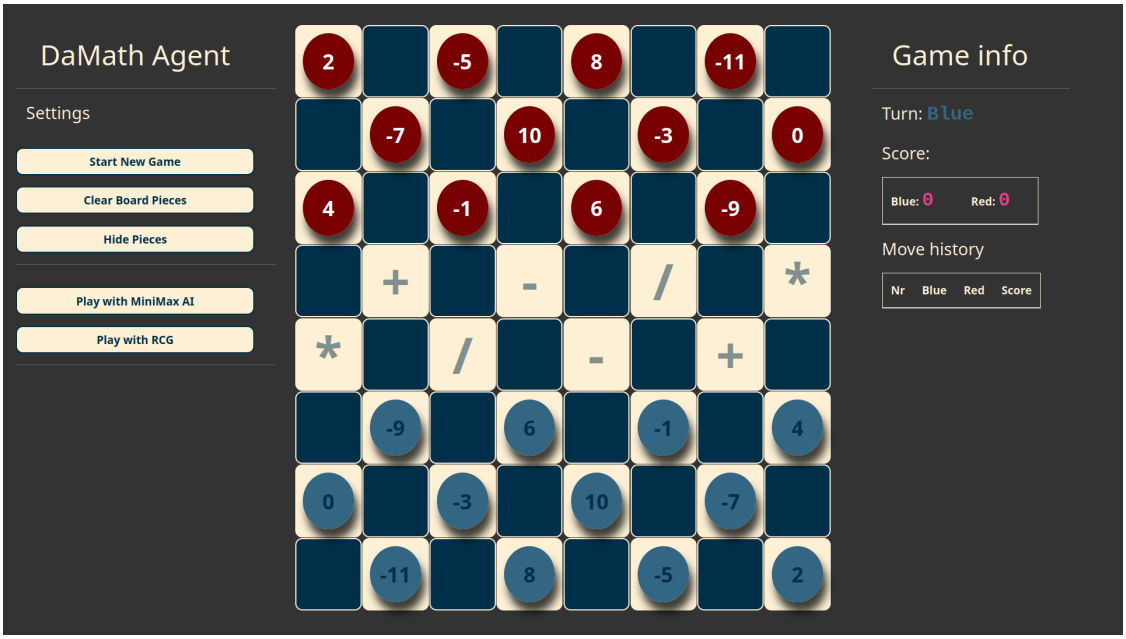


Figure 6.10: Initial state of the game board showing all pieces in their starting positions.

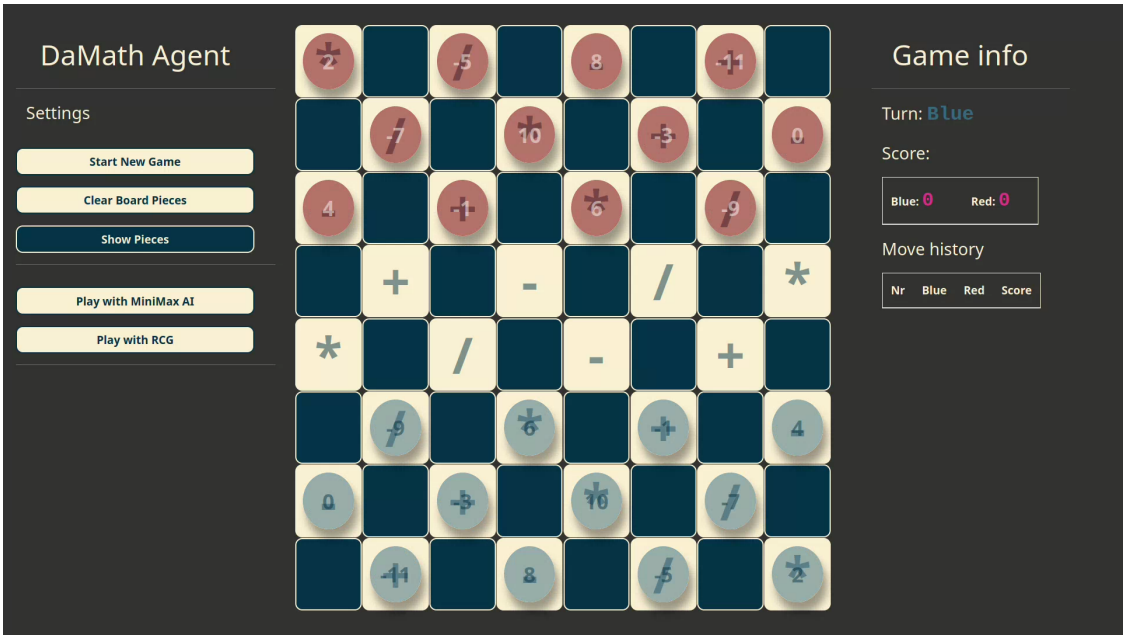


Figure 6.11: Board with pieces hidden for strategy visualization.

6.3.2 | How to Play

The DaMath game follows a turn-based structure between two players, each controlling either the red or blue pieces. Here is a step-by-step guide on how to play using the web application:

1. **Starting the Game:** Upon loading the application, the initial board is displayed with all pieces in their starting positions. Players take turns to move starting with Blue.
2. **Selecting a Piece:** Click on your piece to highlight valid moves. Valid moves are determined by DaMath movement rules—diagonal forward for regular pieces and multi-directional for Dama (kinged) pieces.
3. **Moving a Piece:** Click on a highlighted destination square to move the piece. If a capture is possible, the application will compute the result of the mathematical operation based on the piece values and the operator on the target square.
4. **Capturing a Piece:** When capturing, the attacking piece jumps over the opponent's piece. The operation between the values of the attacking and defending pieces determines the score gained.
5. **Promotion to Dama:** If a regular piece reaches the opponent's baseline, it is promoted to a Dama and gains the ability to move backward and diagonal ends of the board.
6. **Scoring:** After every capture, the application automatically updates the player's score based on the result of the operation.
7. **Reset and Options:** The user can reset the board or toggle display options (e.g., hide/show pieces) using the control panel.
8. **Winning the Game:** The game ends when one player captures all opposing pieces or when no valid moves are left. The player with the higher score wins.

6.4 | Summary

The game-playing performance ranking places the RCG AI at the bottom, the LLM-powered agent in the middle, and the classic MiniMax algorithm at the top. While our decision-making "best-move" prompt enables the LLM agent to make far stronger tactical decisions than RCG—especially when maximizing immediate multi-capture gains—it still falls short of the deeper strategic understanding inherent in MiniMax.

One of the LLM agent's key strengths lies in its ability to identify and execute high-value capture sequences, this adaptive scoring focus represents a clear improvement over the static rule set of RCG. However, the agent exhibits pronounced weaknesses in board awareness and long-term planning. Its decision horizon is essentially myopic: it prioritizes the next capture without adequately considering opponent threats, control of critical diagonals, or end-game positioning. Even though these are established in the prompt instructions, the LLM Agent struggles to follow through these instructions. This short-term planning bias sometimes leads it to sacrifice piece safety for immediate material gain, resulting in avoidable losses when pitted against a deeper search strategy like MiniMax.

7

Conclusions and Recommendations

This chapter covers the study's conclusions as well as recommendations for other implementations or improvisations of the LLM Agent, along with the achieved objectives from the specific objectives outlined in Section 1.3.2.

7.1 | Achieved Objectives

The first challenge in creating the LLM Agent was to develop the game representations in the DaMath board game to be used in the Game Engine and the LLM Agent itself. This was the first objective that was achieved, and it was achieved quite early on. To begin with, the board was represented as a 1-Dimensional array, a Python list object, and a JSON object for abstraction. Finally, a representation of the movement notations is also created as a JSON object, which is used as the LLM Agent's output.

The Damath Game Engine was also established, which allowed game instances to be passed on to the frontend session, and was used in the Random Choice Generator AI and the MiniMax AI players, with the MiniMax AI requiring the game instance as it is a state for the MiniMax AI used to search for beneficial game states.

The most challenging process in achieving all of the objectives was to create the LLM Agent. The LLM Agent is structured with a sequence of prompt instructions. First, the LLM Agent generates code that when given a board state as an input, generates partial valid moves. These partial valid moves are passed on another instruction set that returns a final valid moves, ideally similar to what valid moves the Game Engine provides. These valid moves are then passed on to another instruction set in order for the LLM Agent to determine the best move that it can choose from the given valid

moves. After this was completed, the LLM Agent was ready to be integrated as an LLM-Driven Opponent that can play against humans, and even other AI variations.

The LLM agent was then evaluated by playing against a Random Choice Generator (RCG) AI Player and a MiniMax AI Player. Each AI Player was competing with the LLM Agent for 50 games. Each game was compiled and tabulated, where other forms of data were visualized and then discussed. Additionally, the code generation section was also evaluated by running it 5 times, gathering each instance's runtime in order to get the average runtime.

To finish it all off, the DaMath system was fully established. All the systems such as the LLM Agent, the DaMath Game Engine, the two AI Player variations, are incorporated into a prototype. This prototype also features a user interface where a human player can interact with the system, playing against the LLM Agent. After a game instance ends, the player can choose to play again with the LLM Agent.

7.2 | Critique and Limitations

As expected, the LLM Agent underperforms, since the model that was used has parameters lower than those used on existing capable AI agents such as ChatGPT, Deepseek, and more. There might also be other implementations far better than what was presented in this study.

Another factor for the agent's underperformance might also be due to the prompt instructions themselves. However, it was quite challenging to develop the workflow for the LLM Agent because the model used was having a difficult time following the instructions well. Even when the instructions were separated into smaller tasks, the model was found to struggle with doing simpler tasks.

Lastly, the LLM Agent is not designed to play fast, unlike the standard AI opponent. This is influenced by how many LLM calls are made. In the LLM Agent's workflow, it is designed such that when the agent chooses a best move that is not a valid move in the Game Engine, it reiterates the process of choosing another best move while using a new list of LLM Agent-generated valid moves with the old invalid move choice being popped. In the case of the list of valid moves produced by the LLM Agent being empty, it reiterates the process of generating a new list of valid moves, making the whole process very slow.

7.3 | Future Work

The LLM Agent presented in the study is far from ideal. Further studies on improving the agent are described as follows:

- **Explore different prompt structures:** A recent open-sourced Python package by Meta (2025) converts prompts that are effective with other Large Language Models (LLMs) into versions specifically optimized for LLaMA models. This approach may enhance the model's ability to select valid moves more accurately and consistently provide the best possible move with optimal performance.
- **Leverage another generally pre-trained model:** The LLM Agent's biggest weakness is that there are a lot of times in which the model hallucinates or struggles to follow basic instructions. Leveraging a better pre-trained model for general tasks might boost the LLM Agent's performance by a lot. Choosing a model with more parameters than the one used in the study should also increase the LLM Agent's performance significantly.
- **Utilize a model trained heavily on DaMath information:** A model might be released that is trained with huge amounts of data relating DaMath. This model will perform similarly to how existing models perform on Chess and Checkers. Utilizing this kind of model in the existing LLM Agent might help the agent determine valid moves and the best move easier.

7.4 | Final Remarks

The study successfully explored the potential of leveraging LLMs as game opponents in the board game DaMath, including the previous failed attempts discussed on Chapter 5. An LLM Agent was created, alongside a prototype where a human player can interact with the agent in a game of DaMath. Although the agent outperformed a Random Choice Generator AI, it fared poorly when facing the MiniMax AI. Overall, the LLM's performance was underwhelming, albeit it was still able to play the game from start to finish.

The performance of the LLM agent reveals meaningful potential for future development. Its capacity to learn and adapt based on prompt engineering alone—without access to explicit search trees or game-specific heuristics—suggests that with further refinement, such as integrating memory, planning modules, or fine-tuning

on gameplay data, LLMs could evolve into competent decision-makers in structured environments like Damath. This study does not claim LLMs outperform traditional methods yet, but it does indicate a promising direction: that with the right guidance and tools, LLMs may one day rival or even augment classic AI strategies in turn-based, rule-based games.

A

Resource Persons

Malikey M. Maulana
Thesis Adviser, Computer Science Department
College of Computer Studies
Mindanao State University - Iligan Institute of Technology

B

Personal Vitae

Full Name: Ramel Cary B. Jamen
Residence Address: Camote, Maon, Butuan City, Agusan del Norte, 8600
Contact Number: +63 966 834 0346
Email Address: ramelcary.jamen@g.msuiit.edu.ph

Full Name: Edward Vincent G. Escasio
Residence Address: Luz Village, Butuan City, Agusan del Norte, 8600
Contact Number: +63 945 146 9042
Email Address: edwardvincent.escasio@g.msuiit.edu.ph

Full Name: Gon Vincent C. Alicando
Residence Address: Zone Mars, Brgy. Suarez, Iligan City, Lanao del Norte, 9200
Contact Number: +63 951 837 5617
Email Address: gonvincent.alicando@g.msuiit.edu.ph

C

Prompt Templates

```
normal_moves_instruction = f"""I have an initial board state (A 1-dimensional array indicated as a Python list).
I want you to create a code that takes in a board state as an input with the function name 'func1'.

First, create a Piece class that has the attributes:
1. color: Indicates the piece's color (e.g., "r" for red, "b" for blue).
2. value: The numerical value assigned to that piece.
3. is_dama: A Boolean indicating whether the piece is promoted (dama/king) or not.
4. index: this has a default value of 0. This is the position of a piece on the board_state. This should be assigned during the func1 function call.
5. name: should be a string with the format 'name, value'
A piece's position is determined by its index (0 to 63) in the board list (excluding unusable squares).

The piece should have these dunder methods:
__repr__: it should look like this Piece('r', 2, is_dama=False)
__eq__: it should return a boolean if another object is+ an instance of the Piece class, and their names should be equal
__hash__: the hash of its name

func1 should follow these logic:
1. The function should iterate on each element of the board state. Enumerate the board state so you can get the index and the element.
```

```
2. If the element is a string 'X', do nothing.
3. If the element is a List, check the first element of that list.
4. If the first element is None, do nothing. Else, if it is an instance of the Piece class, AND if the piece's color is 'r', assign an attribute 'index' to the Piece object its index (piece.index = index). Then, call the func2 function, capture its returned key:value pair, and update the valid_moves dictionary with this data.
6. After func2 returns the key:value pair of valid moves, merge this pair into the valid_moves dictionary so that all pieces' moves are recorded.
7. Return the dictionary of all valid moves.

Logic for func2:
1. The function takes in a Piece object.
2. With its index as its position on the board (source index), add 7 to perform a forward left movement, and add 9 to perform a forward right movement, each of these results is a destination index.
3. For each destination index, first check if the index is within the valid range of the board state by ensuring it is less than len(board_state). Then, access the destination square using board_state[destination_index]. If the index is out-of-range, consider the move invalid; otherwise, use it to check the board state.
4. If the destination square is a list, access its first element. If the first element is None, consider it a valid move; otherwise, if it is an instance of the Piece class,or if it is a string 'X', consider it an invalid move.
5. Return a key, value pair of the piece source index with a list of its valid moves ONLY.

Here is the board_state:
{normal_moves_board_state}

Include the board state in your code (Place it after the Piece class is created).
"""
```

Figure C.1: Code Generation - Valid Normal Pieces Moves

```
dama_moves_instruction = f"""I have an initial board state (A 1-dimensional array indicated as a Python list).
I want you to create a code that takes in a board state as an input with the function name 'func1'.

First, create a Piece class that has the attributes:
1. color: Indicates the piece's color (e.g., "r" for red, "b" for blue).
2. value: The numerical value assigned to that piece.
3. is_dama: A Boolean indicating whether the piece is promoted (dama/king) or not.
4. index: this has a default value of 0. This is the position of a piece on the board_state. This should be assigned during the func1 function call.
5. name: should be a string with the format 'name, value'
A piece's position is determined by its index (0 to 63) in the board list (excluding unusable squares).

The piece should have these dunder methods:
__repr__: it should look like this Piece('r', 2, is_dama=False)
__eq__: it should return a boolean if another object is+ an instance of the Piece class, and their names should be equal
__hash__: the hash of its name

func1 should follow these logic:

1. The function should iterate on each element of the board state. Enumerate the board state so you can get the index (i) and the element.
2. If the element is a string 'X', do nothing.
3. If the element is a List, check the first element of that list.
4. If the first element is None, do nothing.
5. Else, if it is an instance of the Piece class, if the piece's color is 'r', AND if the piece is a dama, do these following logic: First, assign the index value to the piece's 'index' attribute. Next, call the func2 function, capture its returned key:value pair. Then, update the valid_moves dictionary with this data.
6. After func2 returns the key:value pair of valid moves, merge this pair into the valid_moves dictionary so that all pieces' moves are recorded.
```

```
7. After the function iterates on all element of the board state, return the dictionary of all valid moves.

Logic for func2:
1. The function also takes in a Piece object.
2. Instantiate a list called damamove with the values [7, 9, -9, -7] and a local empty list called moves.
3. Iterate each element on the damamove list. Call the element 'mv'.
4. For each mv in damamove, set dest_index to the piece's index and initialize an empty list called holder.
5. Use a while True loop to keep adding the move offset (mv) to dest_index and, add a condition if the destination index is within range between 0 and the board_state length, and the destination square is empty (i.e. a list whose first element is None), append that destination index to holder. Stop when the square is occupied or out-of-bounds.
6. After the loop, convert holder into a tuple and append this tuple to moves (do not include the piece's index in the tuple).
7. After processing all move directions, simply return the piece index and the list 'moves'.

Here is the board_state:
{dama_moves_board_state}

Include the board state in your code (Place it after the Piece class is created).
"""
```

Figure C.2: Code Generation - Valid Dama Pieces Moves

```
normal_captures_instruction = f"""I have an initial board state (A 1-dimensional array indicated as a Python list).
I want you to create a code that takes in a board state as an input with the function name 'func5'.

First, create a Piece class that has the attributes:
1. color: Indicates the piece's color (e.g., "r" for red, "b" for blue).
2. value: The numerical value assigned to that piece.
3. is_dama: A Boolean indicating whether the piece is promoted (dama/king) or not.
4. index: this has a default value of 0. This is the position of a piece on the board_state. This should be assigned during the func5 function call.
5. name: should be a string with the format 'name, value'
A piece's position is determined by its index (0 to 63) in the board list (excluding unusable squares).

The piece should have these dunder methods:
__repr__: it should look like this Piece('r', 2, is_dama=False)
__eq__: it should return a boolean if another object is+ an instance of the Piece class, and their names should be equal
__hash__: the hash of its name

func5 should follow these logic:

1. The function should iterate on each element of the board state. Enumerate the board state so you can get the index and the element.
2. If the element is a string 'X', do nothing.
3. If the element is a List, check the first element of that list.
4. If the first element is None, do nothing. Else, if it is an instance of the Piece class, AND if the piece's color is 'r', assign an attribute 'index' to the Piece object its index (piece.index = index).
5. Then, call the func6 function, capture its returned key:value pair, and update the valid_moves dictionary with this data.
6. After func6 returns the key:value pair of valid moves, merge this pair into the valid_moves dictionary so that all pieces' moves are recorded.
```

```
7. Return the dictionary of all valid moves.

Logic for func6:
1. The function takes in a Piece object.
2. Instantiate an empty list called 'capmoves' and a list called 'dia' with the values of 7, 9, -7, -9. Additionally, set a variable named 'src_ind' to the index of the piece object.
3. Iterate all the elements inside dia using a variable 'd'. Inside the iteration, set a variable named 'temp_ind' to have the value of 'src_ind'.
4. Add temp_ind with the variable 'd' from the list dia. Assign it to a variable named 'capt_ind'.
5. Check if capt_ind is between 0 and the length of the board state. If it is not in between, continue.
6. Then, check if board_state[capt_ind] is a list. If it is a list, use a nested if-statement to check if the first element is an instance of the Piece class, and if the piece's color is blue. If this is True, add capt_ind with the value of the variable 'd', and store it to a variable named 'dest_ind'.
7. Use the dest_ind as an index to the board state. Check if it is a list and the first element is None, append the dest_ind to the capmoves list.
8. Return a pair of piece.index and capmoves.

Here is the board_state:
{normal_captures_board_state}

Include the board state in your code (Place it after the Piece class is created).
"""
```

Figure C.3: Code Generation - Valid Normal Pieces Captures

```
dama_captures_instruction = f"""I have an initial board state (A 1-dimensional array indicated as a Python list).
I want you to create a code that takes in a board state as an input with the function name 'func7'.

First, create a Piece class that has the attributes:
1. color: Indicates the piece's color (e.g., "r" for red, "b" for blue).
2. value: The numerical value assigned to that piece.
3. is_dama: A Boolean indicating whether the piece is promoted (dama/king) or not.
4. index: this has a default value of 0. This is the position of a piece on the board_state. This should be assigned during the func7 function call.
5. name: should be a string with the format 'name, value'
A piece's position is determined by its index (0 to 63) in the board list (excluding unusable squares).

The piece should have these dunder methods:
__repr__: it should look like this Piece('r', 2, is_dama=False)
__eq__: it should return a boolean if another object is an instance of the Piece class, and their names should be equal
__hash__: the hash of its name

func7 should follow these logic:
1. The function should iterate on each element of the board state. Enumerate the board state so you can get the index (i) and the element.
2. If the element is a string 'X', do nothing.
3. If the element is a List, check the first element of that list.
4. If the first element is None, do nothing.
5. Else, if it is an instance of the Piece class, if the piece's color is 'r', AND if the piece is a dama: set piece.index to the value of 'i', call the func8 function, capture its returned pair using key, value. Then, update the valid_moves dictionary using the data.
6. After func8 returns the key:value pair of valid moves, merge this pair into the valid_moves dictionary so that all pieces' moves are recorded.
7. After the function iterates on all element of the board state, return the dictionary of all valid moves.
```

```
Logic for func8:
1. In func8, accept a Piece object and initialize an empty list named 'capmoves'.
2. Define a list 'dia' with values [7, 9, -7, -9] and set 'src_ind' to the piece's index.
3. For each value d in dia, initialize 'capt_ind' with the value of src_ind added to d, and an empty list named holder.
4. While capt_ind is between 0 to the length of board_state AND if the board_state with an index of capt_ind is an instance to a list, create an if, elif, else blocks stated on steps 5, 6, 8, respectively.
5. If the first element of the board_state[capt_ind] is None, increment capt_ind with the value of d, then continue.

6. Else, if the first element of the board_state[capt_ind] has an attribute 'color' with the value of a string 'b', set dest_ind with the value of capt_ind added by d.
Inside this elif block, create a while loop with the conditions: dest_ind is between 0 and len(board_state), board_state[dest_ind] is an instance of a list, the first element of the board_state[dest_ind] is None, append dest_ind to the list holder, increment dest_ind with d. Outside the while block, break the outer loop.
7. Else, break out of the loop.
8. After breaking out of the while loop, append a tuple version of the holder list to the list 'capmoves'.
9. After all iterations of the for-loop, return exactly piece.index, capmoves.

Here is the board_state:
{dama_captures_board_state}

Include the board state in your code (Place it after the Piece class is created).
"""
```

Figure C.4: Code Generation - Valid Dama Pieces Captures

```
move_template = ChatPromptTemplate.from_template("""
You are given a dictionary named "results" that contains only two keys: "normal_moves" and "dama_moves".

Each of these keys maps to a dictionary:
- In "normal_moves", keys are integers, and values are lists of integers.
- In "dama_moves", keys are integers, and values are lists of tuples of integers.

Your task is:
1. Ignore the top-level keys ("normal_moves" and "dama_moves") and work only with their inner dictionaries.
2. Merge the two inner dictionaries into one:
    - For each shared key, keep the value from the "dama_moves".
    - Flatten all tuples in "dama_moves" values into one list of integers before merging.
    - If a key only exists in one of the dictionaries, use its value directly.
3. The final result should be a JSON object with a single key "moves", whose value is the merged dictionary.
4. All keys in the final dictionary should be strings.

Here is the results dictionary:
{results}

Return only the final JSON with key "moves".
""")
```

Figure C.5: Results to Valid Moves

```
capture_template = ChatPromptTemplate.from_template("""
You are given a dictionary named "results" that may contain either one of these two keys: "normal_captures" and "dama_captures".

Each of these keys maps to a dictionary:
- In "normal_captures", keys are integers, and values are lists of integers.
- In "dama_captures", keys are integers, and values are lists of tuples of integers.

Your task is:
1. Ignore the top-level keys and work only with the values of the inner dictionaries.
2. Choose one of the inner dictionaries:
    - If "dama_captures" exists **and** is not empty, flatten the tuples in its values (i.e., convert each list of tuples into a list of integers), and use this dictionary.
    - Otherwise, use the "normal_captures" dictionary.
3. The final result should be a JSON object with a single key "captures", whose value is the selected and possibly modified dictionary.
4. All keys in the final dictionary should be strings.

Here is the results dictionary:
{results}

Return only the final JSON with key "captures".
""")
```

Figure C.6: Results to Valid Captures

```
best_move_prompt = ChatPromptTemplate.from_template(
"""System Prompt:
You are a Damath game-playing agent that understands the board state representation.
The game state is provided as a JSON format where the value is a list of square objects.
Each square object has:
    - \"position\": a two-element list [index, operator], where index is an element of [0,63] is the board coordinate in row-major order and operator is an element of {{'*','/','-','+'}} denotes the arithmetic operator on that square.
    - \"piece\": either null if empty, or a three-element list [color, value, is_dama], where:
        * color is an element of {{'red','blue'}}
        * value is an integer (positive or negative) representing the piece's numeric value
        * is_dama is a boolean indicating king status

The valid_moves list is a source-destination list with a pair indices [[source, dest], [source, dest]].

**Normal Move Priorities:**
1. Advancing toward the opponent's back rank (especially pieces near promotion at index 63).
2. Protecting high-value pieces (value indicates risk level).
3. Controlling central or strategic squares.
4. Setting up future captures or blocking opponent runs.

**Dama/King Considerations:**
- Use Dama moves only if a clear positional or material advantage outweighs a normal advance.
- Damas may traverse multiple empty squares; simulate landing spots for positioning.

**Strategic Layer:**
- Threat Analysis: Avoid leaving the moved piece immediately capturable.
- Multi-Ply Forecast: Internally look 2-3 plies ahead to avoid traps.
- Balance material gain vs. positional strength and promotion potential.
```

```
**Output:**
Return only a JSON object with keys:
{{
\"source\": <int>,
\"destination\": <int>,
\"reason\": <string>
}}
The reason should reference positional strategy (advancement,
↪ protection, control).

Your turn-select the optimal normal move and output JSON only.
"""
)
```

Figure C.7: Valid Moves to Best Move

```
best_capture_prompt = ChatPromptTemplate.from_template(
"""System Prompt:
You are a Damath game-playing agent that understands the board
↪ state representation.
The game state is provided as a JSON format where the value is a
↪ list of square objects.
Each square object has:
    - \"position\": a two-element list [index, operator], where
    ↪ index is an element of [0,63] is the board coordinate in
    ↪ row-major order and operator is an element of
    ↪ {{'*','/','-','+'}} denotes the arithmetic operator on that
    ↪ square.
    - \"piece\": either null if empty, or a three-element list
    ↪ [color, value, is_dama], where:
        * color is an element of {{'red','blue'}}
        * value is an integer (positive or negative) representing
        ↪ the piece's numeric value
        * is_dama is a boolean indicating king status

Choose from the valid_moves list maps source positions to a list of
↪ triples [source, destination, score].
```

```
**Capture Selection Rules:**
1. Normal captures occur at offsets +14, -14, +18, -18. After moving to a capture destination, assess whether additional captures are possible from that new square.
2. Automatically choose the single capture with the highest positive score, avoid negative scores you will lose some points.
3. For Dama pieces, consider multi-step capture chains: captures may still occur at offsets ±14 and ±18, including longer jumps where the offset is a multiple of 14 (i.e., 7*2) or 18 (i.e., 9*2) to maximize total score.
4. Perform full internal chain-of-thought; do not reveal it.

**Output:**
Return only a JSON object with keys:
{{
\"source\": <int>,
\"destination\": <int>,
\"reason\": <string>
}}
The reason should reference the capture score and the sequence rationale.

Your turn-select the best capture and output JSON only.
"""
)
```

Figure C.8: Valid Moves to Best Capture

D

Algorithm Implementation

Code Generation - Valid Move Utilities

```
import sys, re, time
from langchain_ollama import ChatOllama

start = time.time()

class Piece:
    def __init__(self, color, value, is_dama=False):
        self.value = value
        self.index = None
        self.color = color
        self.is_dama = is_dama
        self.name = f"{color}, {value}"

    def __repr__(self):
        return f"Piece({self.name}, is_dama={self.is_dama})"

    def __eq__(self, other):
        return isinstance(other, Piece) and self.name == other.name

    def __hash__(self):
        return hash(self.name)

normal_moves_board_state = [[Piece('b', 2, is_dama=False), '*'],
↪  'X', [Piece('r', -5, is_dama=False), '/'], 'X', [Piece('r', 8,
↪  is_dama=False), '-'], 'X', [Piece('r', -11, is_dama=False),
↪  '+'], 'X', 'X', [Piece('r', -7, is_dama=False), '/'], 'X',
```

```
23  [Piece('r', 10, is_dama=False), '*'], 'X', [Piece('r', -3,
    ↪  is_dama=False), '+'], 'X', [Piece('r', 0, is_dama=False), '-'],
    ↪  [Piece('r', 4, is_dama=False), '-'], 'X', [Piece('r', -1,
    ↪  is_dama=False), '+'], 'X', [Piece('r', 6, is_dama=False), '*'],
    ↪  'X', [Piece('r', -9, is_dama=False), '/'], 'X', 'X', [None,
    ↪  '+'], 'X', [None, '-'], 'X', [None, '/'], 'X', [None, '*'],
    ↪  [None, '*'], 'X', [None, '/'], 'X', [None, '-'], 'X', [None,
    ↪  '+'], 'X', 'X', [Piece('b', -9, is_dama=False), '/'], 'X',
    ↪  [Piece('b', 6, is_dama=False), '*'], 'X', [Piece('b', -1,
    ↪  is_dama=False), '+'], 'X', [Piece('b', 4, is_dama=False), '-'],
    ↪  [Piece('b', 0, is_dama=False), '-'], 'X', [Piece('b', -3,
    ↪  is_dama=False), '+'], 'X', [Piece('b', 10, is_dama=False),
    ↪  '*'], 'X', [Piece('b', -7, is_dama=False), '/'], 'X', 'X',
    ↪  [Piece('b', -11, is_dama=False), '+'], 'X', [Piece('b', 8,
    ↪  is_dama=False), '-'], 'X', [Piece('b', -5, is_dama=False),
    ↪  '/'], 'X', [Piece('b', 2, is_dama=False), '*']]
24
25  dama_moves_board_state = [[None, '*'], 'X', [None, '/'], 'X',
    ↪  [None, '-'], 'X', [None, '+'], 'X', 'X', [None, '/'], 'X',
    ↪  [None, '*'], 'X', [Piece('r', -11, is_dama=False), '+'], 'X',
    ↪  [Piece('r', 0, is_dama=False), '-'], [None, '-'], 'X', [None,
    ↪  '+'], 'X', [Piece('r', 6, is_dama=True), '*'], 'X', [Piece('r',
    ↪  -9, is_dama=False), '/'], 'X', 'X', [None, '+'], 'X', [None,
    ↪  '-'], 'X', [None, '/'], 'X', [None, '*'], [Piece('b', 0,
    ↪  is_dama=True), '*'], 'X', [None, '/'], 'X', [None, '-'], 'X',
    ↪  [None, '+'], 'X', 'X', [None, '/'], 'X', [None, '*'], 'X',
    ↪  [None, '+'], 'X', [None, '-'], [Piece('r', -5, is_dama=True),
    ↪  '-'], 'X', [None, '+'], 'X', [None, '*'], 'X', [None, '/'],
    ↪  'X', 'X', [None, '+'], 'X', [None, '-'], 'X', [None, '/'], 'X',
    ↪  [None, '*']]
26
27  normal_captures_board_state = [[Piece('r', -112, is_dama=False),
    ↪  '*'], 'X', [None, '/'], 'X', [None, '-'], 'X', [None, '+'],
    ↪  'X', 'X', [Piece('b', 0, is_dama=False), '/'], 'X', [None,
    ↪  '*'], 'X', [None, '+'], 'X', [None, '-'], [None, '-'], 'X',
    ↪  [None, '+'], 'X', [None, '*'], 'X', [Piece('r', -9,
    ↪  is_dama=False), '/'], 'X', 'X', [None, '+'], 'X', [Piece('b',
    ↪  -11, is_dama=False), '-'], 'X', [None, '/'], 'X', [None, '*'],
    ↪  [Piece('b', 0, is_dama=False), '*'], 'X', [Piece('r', -5,
    ↪  is_dama=True), '/'], 'X', [None, '-'], 'X', [None, '+'], 'X',
    ↪  'X', [None, '/'], 'X', [Piece('b', -5, is_dama=True), '*'],
    ↪  'X', [None, '+'], 'X', [None, '-'], [None, '-'], 'X',
```

```
27  [None, '+'], 'X', [None, '*'], 'X', [Piece('b', 6, is_dama=False),
    ↪  '/'], 'X', 'X', [None, '+'], 'X', [None, '-'], 'X', [None,
    ↪  '/'], 'X', [Piece('r', 6, is_dama=False), '*']]
28
29  dama_captures_board_state = [[Piece('r', -112, is_dama=False),
    ↪  '*'], 'X', [None, '/'], 'X', [None, '-'], 'X', [None, '+'],
    ↪  'X', 'X', [Piece('b', 0, is_dama=False), '/'], 'X', [None,
    ↪  '*'], 'X', [None, '+'], 'X', [None, '-'], [None, '-'], 'X',
    ↪  [None, '+'], 'X', [None, '*'], 'X', [Piece('r', -9,
    ↪  is_dama=False), '/'], 'X', 'X', [None, '+'], 'X', [Piece('b',
    ↪  -11, is_dama=False), '-'], 'X', [None, '/'], 'X', [None, '*'],
    ↪  [Piece('b', 0, is_dama=False), '*'], 'X', [Piece('r', -5,
    ↪  is_dama=True), '/'], 'X', [None, '-'], 'X', [None, '+'], 'X',
    ↪  'X', [None, '/'], 'X', [Piece('b', -5, is_dama=True), '*'],
    ↪  'X', [None, '+'], 'X', [None, '-'], [None, '-'], 'X', [None,
    ↪  '+'], 'X', [None, '*'], 'X', [Piece('b', 6, is_dama=False),
    ↪  '/'], 'X', 'X', [None, '+'], 'X', [None, '-'], 'X', [None,
    ↪  '/'], 'X', [Piece('r', 6, is_dama=False), '*']]
30
31  def normal_moves_runLLM(instruction):
32      llm = ChatOllama(model="llama3.2:3b-instruct-q8_0",
        ↪  temperature=0.5)
33      chain = llm
34      res = ""
35      res = chain.invoke(instruction)
36      return res
37
38  def dama_moves_runLLM(instruction):
39      llm = ChatOllama(model="llama3.2:3b-instruct-q8_0",
40          temperature=0.5)
41      chain = llm
42      res = ""
43      res = chain.invoke(instruction)
44      return res
45
46  def normal_captures_runLLM(instruction):
47      llm = ChatOllama(model="llama3.2:3b-instruct-q8_0",
        ↪  temperature=0.5)
48      chain = llm
```

```
    res = ""
    res = chain.invoke(instruction)
    return res

def dama_captures_runLLM(instruction):
    llm = ChatOllama(
        model="llama3.2:3b-instruct-q8_0",
        temperature=0.5,
    )
    chain = llm
    res = ""
    res = chain.invoke(instruction)
    return res


def formatOutput(response, name):
    match = re.search(r"```(?:python)?\n(.*?)```", response,
    ↪  re.DOTALL)
    if match:
        python_code = match.group(1)

        with open(f"./utilities/{name}.py", "w") as file:
            file.write(python_code)

        print(f"Python code extracted and saved to
        ↪  './utilities/{name}.py'.")
    else:
        print("No Python code block found in the provided output.")


def generate_normal_moves():
    print("Generating normal moves...")
    normal_moves = normal_moves_runLLM(normal_moves_instruction)
    normal_moves = normal_moves.content
    formatOutput(normal_moves, "normal_moves")
    time.sleep(10)

def generate_dama_moves():
    print("Generating dama moves...")
    dama_moves = dama_moves_runLLM(dama_moves_instruction)
    dama_moves = dama_moves.content
    formatOutput(dama_moves, "dama_moves")
    time.sleep(10)
```

```
90  def generate_normal_captures():
91      print("Generating normal captures...")
92      normal_captures =
        ↪  normal_captures_runLLM(normal_captures_instruction)
93      normal_captures = normal_captures.content
94      formatOutput(normal_captures, "normal_captures")
95      time.sleep(10)
96
97  def generate_dama_captures():
98      print("Generating dama captures...")
99      dama_captures = dama_captures_runLLM(dama_captures_instruction)
100     dama_captures = dama_captures.content
101     formatOutput(dama_captures, "dama_captures")
102
103 def main():
104     start = time.time()
105     generators = {
106         "normal_moves": generate_normal_moves,
107         "dama_moves": generate_dama_moves,
108         "normal_captures": generate_normal_captures,
109         "dama_captures": generate_dama_captures
110     }
111     if len(sys.argv) == 1:
112         for generator in generators.values():
113             generator()
114     elif len(sys.argv) == 2:
115         generator_type = sys.argv[1]
116         if generator_type in generators:
117             print(f"Running {generator_type} generator...")
118             generators[generator_type]()
119         else:
120             print(f"Unknown generator type: {generator_type}")
121             print("Available types:", list(generators.keys()))
122     else:
123         [normal_moves|dama_moves|normal_captures|dama_captures]")
124
125     end = time.time()
126     print("It took", end-start, "seconds to run.")
127
128 if __name__ == "__main__":
129     main()
```

Code Generation - Test Cases Shell Script

```
#!/bin/bash

# Function to check if arrays are equal
check_output() {
    expected="$1"
    actual="$2"
    test_name="$3"

    if [ "$actual" = "$expected" ]; then
        echo "$test_name test passed"
        echo "Expected: $expected"
        echo "Got: $actual"
        return 0
    else
        echo "$test_name test failed"
        echo "Expected: $expected"
        echo "Got: $actual"
        return 1
    fi
}

# Expected outputs
EXPECTED_NORMAL_MOVES="{0: [], 2: [], 4: [], 6: [], 9: [], 11: [], 13: [], 15: [], 16: [25], 18: [25, 27], 20: [27, 29], 22: [29, 31]}"
EXPECTED_DAMA_MOVES="{20: [(27, 34, 41), (29, 38, 47), (11, 2)], 48: [(57,), (41, 34, 27)]}"
EXPECTED_DAMA_MOVES_ALT="{20: [(27, 34, 41), (29, 38, 47), (11, 2), ()], 48: [(), (57,), (), (41, 34, 27)]}"
EXPECTED_NORMAL_CAPTURES="{0: [18], 22: [], 34: [52, 20], 63: [45]}"
EXPECTED_DAMA_CAPTURES="{34: [(), (52, 61), (20, 13, 6), ()]}"

# Create test runner
cat > ./utilities/test-code-output.py << 'EOF'
import sys
import signal
from contextlib import contextmanager
from multiprocessing import Process, Queue
import platform
```

```
class TimeoutException(Exception): pass

@contextmanager
def timeout(seconds):
    def signal_handler(signum, frame): raise
    ↪ TimeoutException("Timed out!")
    signal.signal(signal.SIGALRM, signal_handler)
    signal.alarm(seconds)
    try: yield
    finally: signal.alarm(0)

def run_with_timeout(func, args, timeout_seconds):
    q = Queue()
    p = Process(target=lambda: q.put(func(*args)))
    p.start()
    p.join(timeout_seconds)
    if p.is_alive():
        p.terminate()
        p.join()
        raise TimeoutException("Timed out!")
    return q.get()

def safe_run_test(test_name):
    try:
        if test_name == "normal_moves":
            from normal_moves import func1, board_state
            return str(func1(board_state))
        elif test_name == "dama_moves":
            from dama_moves import func1, board_state
            return str(func1(board_state))
        elif test_name == "normal_captures":
            from normal_captures import func5, board_state
            return str(func5(board_state))
        elif test_name == "dama_captures":
            from dama_captures import func7, board_state
            return str(func7(board_state))
    except Exception as e:
        return f"Error: {str(e)}"
```

```
def run_specific_test(test_name):
    MAX_EXECUTION_TIME = 5
    try:
        if platform.system() != 'Windows':
            with timeout(MAX_EXECUTION_TIME):
                result = safe_run_test(test_name)
        else:
            result = run_with_timeout(safe_run_test, (test_name,),
            ↪  MAX_EXECUTION_TIME)

        print(f"{test_name.upper()}:{result}")
    except TimeoutException:
        print(f"{test_name.upper()}:TIMEOUT")
    except Exception as e:
        print(f"{test_name.upper()}:ERROR - {str(e)}")

if __name__ == "__main__":
    for test in sys.argv[1:]:
        run_specific_test(test)
EOF

declare -A EXPECTED_RESULTS=(
    ["normal_moves"]="$EXPECTED_NORMAL_MOVES"
    ["dama_moves"]="$EXPECTED_DAMA_MOVES|$EXPECTED_DAMA_MOVES_ALT"
    ["normal_captures"]="$EXPECTED_NORMAL_CAPTURES"
    ["dama_captures"]="$EXPECTED_DAMA_CAPTURES"
)

declare -A test_status
for test in "${!EXPECTED_RESULTS[@]}"; do
    test_status[$test]=false
done

generate_tests() {
    for test in "${!test_status[@]}"; do
        if ! ${test_status[$test]}; then
            echo "Regenerating $test..."
            python3 ./utilities/code-generator-langchain.py "$test"
            ↪  &
        fi
    done
    wait
}
```

```
114 run_tests() {
115     echo "Running tests..."
116     output=$(python3 ./utilities/test-code-output.py
        ↪  "${!test_status[@]}" | grep -E "^[A-Z_]+:")
117
118     while IFS= read -r line; do
119         test_name=$(echo "$line" | cut -d: -f1 | tr '[:upper:]'
            ↪  '[:lower:]')
120         result=$(echo "$line" | sed "s/^[^:]*://")
121         expected="${EXPECTED_RESULTS[$test_name]}"
122
123         if [[ "$expected" == *"|"* ]]; then
124             IFS="|" read -ra options <<< "$expected"
125             match=false
126             for opt in "${options[@]}"; do
127                 [[ "$result" == "$opt" ]] && match=true && break
128             done
129             if $match; then
130                 echo "$test_name test passed"
131                 test_status[$test_name]=true
132             else
133                 echo "$test_name test failed"
134             fi
135         else
136             if [[ "$result" == "$expected" ]]; then
137                 echo "$test_name test passed"
138                 test_status[$test_name]=true
139             else
140                 echo "$test_name test failed"
141             fi
142         fi
143     done <<< "$output"
144 }
```

```
# Main loop
while true; do
    # Run tests first
    run_tests

    failed_count=0
    for status in "${test_status[@]}"; do
        if ! $status; then
            ((failed_count++))
        fi
    done

    echo "Failed tests: $failed_count"

    if [ $failed_count -eq 0 ]; then
        echo "All tests passed!"
        break
    fi

    # Only generate new code if tests failed
    echo "Generating new code for failed tests..."
    generate_tests

    echo "Retrying in 2 seconds..."
    sleep 2
done
```

Main LLM Agent Workflow

```python
import json, subprocess
from fastapi import FastAPI
from pydantic import BaseModel, Field, ConfigDict
from typing import Dict, List
from langchain_ollama import ChatOllama

last_board_state = None
switch = False
temperature = 0
valid_choice = False
valid_choice_pairs = []
app = FastAPI()

FLASK_BASE_URL = "http://127.0.0.1:5000"

# --- Data Models and Helper Classes ---
class Piece:
    def __init__(self, color, value, is_dama=0, index=0, name=""):
        self.color = color
        self.value = value
        self.is_dama = is_dama
        self.index = index
        self.name = f"{color}, {value}"

    def __repr__(self):
        return f"Piece('{self.color}', {self.value},
        ↪  is_dama={self.is_dama})"

    def __eq__(self, other):
        if not isinstance(other, Piece):
            return False
        return self.name == other.name

    def __hash__(self):
        return hash(self.name)
```

```python
# --- Request Models ---
class StartRequest(BaseModel):
    start: bool

class BoardRequest(BaseModel):
    board: str
    jsonboard: str

# --- Output Schema ---
class MovesSchema(BaseModel):
    moves: Dict[int, List[int]] = Field(
        ..., description="Mapping from key to list of values"
    )

class CapturesSchema(BaseModel):
    captures: Dict[int, List[int]] = Field(
        ..., description="Mapping from key to list of values. These
        ↪  should come from either 'normal_captures' or
        ↪  'dama_captures'"
    )
    model_config = ConfigDict(extra="forbid")


# --- Utility Functions ---
def reinitialize_board(board_state, new_piece_class):
    new_board = []
    for cell in board_state:
        if isinstance(cell, list):
            if cell[0] is None:
                new_board.append([None, cell[1]])
            else:
                old_piece = cell[0]
                new_piece = new_piece_class(
                    color=old_piece.color,
                    value=old_piece.value,
                    is_dama=old_piece.is_dama,
                )
                new_board.append([new_piece, cell[1]])
        else:
            new_board.append(cell)
    return new_board
```

```
74  def get_valid_moves(test_name, board_state):
75      try:
76          if test_name == "normal_moves":
77              from utilities.normal_moves import func1, Piece as
                ↪  NM_Piece
78              import utilities.normal_moves
79              new_board = reinitialize_board(board_state, NM_Piece)
80              utilities.normal_moves.board_state = new_board
81              result =  func1(new_board)
82          elif test_name == "dama_moves":
83              from utilities.dama_moves import func1, Piece as
                ↪  DM_Piece
84              import utilities.dama_moves
85              new_board = reinitialize_board(board_state, DM_Piece)
86              utilities.dama_moves.board_state = new_board
87              result =  func1(new_board)
88          elif test_name == "normal_captures":
89              from utilities.normal_captures import func5, Piece as
                ↪  NC_Piece
90              import utilities.normal_captures
91              new_board = reinitialize_board(board_state, NC_Piece)
92              utilities.normal_captures.board_state = new_board
93              result =  func5(new_board)
94          elif test_name == "dama_captures":
95              from utilities.dama_captures import func7, Piece as
                ↪  DC_Piece
96              import utilities.dama_captures
97              new_board = reinitialize_board(board_state, DC_Piece)
98              utilities.dama_captures.board_state = new_board
99              result =  func7(new_board)
100         return result
101     except Exception as e:
102         return f"Error: {str(e)}"
103
104 def clean_dict(data):
105     if isinstance(data, dict):
106         cleaned = {}
107         for key, value in data.items():
108             cleaned_value = clean_dict(value)
109             if cleaned_value:
110                 cleaned[key] = cleaned_value
111         return cleaned
```

```
112     elif isinstance(data, list):
113         cleaned_list = [clean_dict(item) for item in data if item
            ↪  not in ([], ())]
114         return [item for item in cleaned_list if item != {}]
115     elif isinstance(data, tuple):
116         cleaned_tuple = tuple(item for item in data if item not in
            ↪  ([], ()))
117         return cleaned_tuple if cleaned_tuple else None
118     return data
119 
120 # --- Endpoints ---
121 @app.post("/generate_code")
122 def generate_code(request: StartRequest):
123     if request.start:
124         try:
125             subprocess.run(['bash', 'utilities/script.sh'])
126             return {"status": "completed"}
127         except Exception as e:
128             return {"status": "error", "error": str(e)}
129     else:
130         return {"status": "waiting"}
131 
132 @app.post("/board_to_move")
133 def board_to_move(request: BoardRequest):
134     global last_board_state, switch, temperature, valid_choice,
        ↪  valid_choice_pairs
135     try:
136         board_state = eval(request.board)
137     except Exception as e:
138         return {"status": "error", "error": f"Invalid board format:
            ↪  {str(e)}"}
139 
140     results = {}
141 
142     if not valid_choice or not (last_board_state == board_state):
143         for test in ["normal_moves", "dama_moves",
            ↪  "normal_captures", "dama_captures"]:
144             results[test] = get_valid_moves(test, board_state)
145 
146         results = clean_dict(results)
```

```
147         if "dama_captures" in results.keys() or "normal_captures"
            ↪  in results.keys():
148             is_capture = True
149         else:
150             is_capture = False
151 
152         if not (last_board_state == board_state):
153             switch = False
154         else:
155             temperature += 0.04
156             if temperature > 1:
157                 temperature = 0
158         while True:
159 
160             switch = not switch
161             try:
162                 move_llm_3_1 = ChatOllama(
163                     model="llama3.1:8b-instruct-fp16",
164                     temperature=temperature,
165                     format=MovesSchema.model_json_schema(),
166                 )
167                 move_llm_3_2 = ChatOllama(
168                     model="llama3.2:3b-instruct-fp16",
169                     temperature=temperature,
170                     format=MovesSchema.model_json_schema(),
171                 )
172 
173                 capture_llm_3_1 = ChatOllama(
174                     model="llama3.1:8b-instruct-fp16",
175                     temperature=temperature,
176                     format=CapturesSchema.model_json_schema(),
177                 )
178                 capture_llm_3_2 = ChatOllama(
179                     model="llama3.2:3b-instruct-fp16",
180                     temperature=temperature,
181                     format=CapturesSchema.model_json_schema(),
182                 )
183 
184                 if is_capture:
185                     if switch:
186                         capture_chain = capture_template |
                            ↪  capture_llm_3_1
```

```
                    else:
                        capture_chain = capture_template |
                        ↪ capture_llm_3_2

                    response = capture_chain.invoke({
                        "results": results
                    })

                else:
                    if switch:
                        move_chain: dict = move_template |
                        ↪ move_llm_3_1
                    else:
                        move_chain: dict = move_template |
                        ↪ move_llm_3_2

                    response = move_chain.invoke({
                        "results": results,
                    })

                filtered_results = response.content
                final_results = json.loads(filtered_results)
                valid_moves = final_results

                src_dest_pairs = []

                for key, value in valid_moves.items():
                    value = {int(k): eval(v) if isinstance(v, str)
                    ↪ else eval(str([eval(str(i)) for i in v]))
                    ↪ for k, v in value.items()}

                    if key == 'captures':
                        is_capture = True
                    for source, destinations in value.items():
                        for destination in destinations:
                            if isinstance(destination, tuple) or
                            ↪ isinstance(destination, list):
                                for item in destination:
                            src_dest_pairs.append([source,item])
                            else:
                        src_dest_pairs.append([source,destination])
```

```
                if src_dest_pairs != [] and is_capture:
                    for index, (source, dest) in enumerate(src_dest_pairs):
                        distance = dest - source
                        directions = [-7, -9, 7, 9]
                        for direction in directions:
                            if distance % direction == 0:
                                factor = distance // direction
                                if factor < 0:
                                    direction = abs(direction)
                                break

                        enemy=False
                        middle = source
                        while middle != dest:
                            middle += direction
                            if isinstance(board_state[middle][0], Piece):
                                if board_state[middle][0].color == 'b':
                                    enemy = True
                                    break
                        if enemy:
                            try:
                                srcval = board_state[source][0].value
                                midval = board_state[middle][0].value
                                destop = board_state[dest][1]
                                destop = "//" if destop == "/" else destop
                                score = round(eval(f"{srcval}{destop}{midval}"))
                                capturing_is_dama = board_state[source][0].is_dama
                                captured_is_dama = board_state[middle][0].is_dama
                                if capturing_is_dama and captured_is_dama:
                                    score *= 4
                                elif capturing_is_dama or captured_is_dama:
                                    score *= 2
```

```
                            except ZeroDivisionError:
                                score = 0

                            src_dest_pairs[index] = (source, dest,
                            ↪  score)
                    elif src_dest_pairs == []:
                        raise Exception("Sorry, empty
                        ↪  source-destination pairs")
                    break
                except Exception as e:
                    temperature += 0.04
                    if temperature > 1:
                        temperature = 0
                    else:
                        continue

        last_board_state = board_state
        valid_choice_pairs = src_dest_pairs

    while valid_choice_pairs != []:
        llm_best_choice = ChatOllama(
            model="llama3.1:8b-instruct-fp16",
            temperature=.5,
            format="json"
        )

        is_capture = False
        for i in valid_choice_pairs:
            if isinstance(i, tuple):
                is_capture = True

        if is_capture:
            chain = best_capture_prompt + user_prompt |
            ↪  llm_best_choice
        else:
            chain = best_move_prompt + user_prompt |
            ↪  llm_best_choice

        response = chain.invoke({
        "board_state": request.jsonboard,
        "valid_moves": valid_choice_pairs,
        })
```

```
        response = json.loads(response.content)
        source = response['source']
        destination = response['destination']

        valid_choice = False

        temperature += 0.06
        if temperature > 1:
            temperature = 0
        if valid_choice_pairs == []:
            source = 1
            destination = 1
            valid_choice = False
            break

        for index, item in enumerate(valid_choice_pairs):
            if [item[0], item[1]] == [source, destination]:
                valid_choice = True
                break

        if not valid_choice:
            continue
        else:
            break

    if valid_choice_pairs == [] and valid_choice == True:
        source = 1
        destination = 1
        valid_choice = False
    elif valid_choice_pairs != []:
        valid_choice_pairs.pop(index)

    return {"source": source, "destination": destination}
```

References

AI@Meta. Llama 3.1 model card. 2024. URL https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md.

Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. Playing repeated games with large language models. *ArXiv*, abs/2305.16867, 2023. URL https://api.semanticscholar.org/CorpusID:258947115.

P. Ammanabrolu, W. Broniec, A. Mueller, J. Paul, and M. Riedl. Toward automated quest generation in text-adventure games. *In Proceedings of the International Conference on Computational Creativity (ICCC-20)*, 2020. doi: https://doi.org/10.48550/arXiv.1909.06283.

A. Bakhtin, N. Brown, E. Dinan, and G. Farina. Human-level play in the game of diplomacy by combining language models with strategic reasoning. *Science*, 2022. doi: https://www.science.org/doi/10.1126/science.ade9097.

J. Bantilan, J. Förster, Marinay R. J., and C.A. Rotea. Damath: A 2d educational mobile board game. *International Symposium on Computing for Education*, 2014.

Gabriella A. B. Barros, Leonardo F. B. S. Carvalho, Vitor R. M. Silva, and Roberta V. V. Lopes. An application of genetic algorithm to the game of checkers. In *2011 Brazilian Symposium on Games and Digital Entertainment*, pages 63–69, 2011. doi: 10.1109/SBGAMES.2011.14.

Jan Dirk Blom. A dictionary of hallucinations — link.springer.com. https://link.springer.com/book/10.1007/978-3-031-25248-8, 2023. [Accessed 30-04-2025].

Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890, 2019.

Murray Campbell, A.Joseph Hoane, and Feng hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1): 57–83, 2002. ISSN 0004-3702. doi: https://doi.org/10.1016/S0004-3702(01)00129-1. URL https://www.sciencedirect.com/science/article/pii/S0004370201001291.

Guillaume Chaslot, Jahn-Takeshi Saito, Bruno Bouzy, JWHM Uiterwijk, and H Jaap Van Den Herik. Monte-carlo strategies for computer go. In *Proceedings of the 18th BeNeLux Conference on Artificial Intelligence, Namur, Belgium*, pages 83–91, 2006.

Guillaume Chaslot, Sander Bakkes, Istvan Szita, and Pieter Spronck. Monte-carlo tree search: A new framework for game ai. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 4(1):216–217, Sep. 2021. doi: 10.1609/aiide.v4i1.18700. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/18700`.

Kumar Chellapilla and David B Fogel. Evolving neural networks to play checkers without relying on expert knowledge. *IEEE transactions on neural networks*, 10(6):1382–1391, 1999.

Z. Chu, S. Ni, Z. Wang, X. Feng, M. Yang, and W. Zhang. History, development, and principles of large language models-an introductory survey. *Computation and Language (cs.CL)*, 2024. doi: https://doi.org/10.48550/arXiv.2402.06853.

Sarah Chudleigh. Complete guide to llm agents (2025). `https://botpress.com/blog/llm-agents`, 2025. [Accessed 02-05-2025].

Matthew Ciolino, Josh Kalin, and David Noever. The go transformer: Natural language modeling for game play. 2020. URL `https://arxiv.org/pdf/2007.03500`.

Daniel Crha. Board game with artificial intelligence. 2020.

Sittie Maisah-Nehar M. Dalidig. Effects of game-based teaching on students' conceptual understanding. *6th International Conference on Education and Technology (ICET)*, 2020. doi: https://typeset.io/papers/effects-of-game-based-teaching-on-students-conceptual-4p3qlsh09z.

Omid E David, Nathan S Netanyahu, and Lior Wolf. Deepchess: End-to-end deep neural network for automatic learning in chess. In *Artificial Neural Networks and Machine Learning–ICANN 2016: 25th International Conference on Artificial Neural Networks, Barcelona, Spain, September 6-9, 2016, Proceedings, Part II 25*, pages 88–96. Springer, 2016.

Dep-Ed. Damath: learning math the pinoy way. *Office of the Secretary*, 2010. URL `https://web.archive.org/web/20100307031352/http://www.deped.gov.ph/cpanel/uploads/issuanceImg/feb22-damath.pdf`.

Elmer R. Escandon and Joseph Campion. Minimax checkers playing gui: A foundation for ai applications. In *2018 IEEE XXV International Conference on Electronics, Electrical Engineering and Computing (INTERCON)*, pages 1–4, 2018. doi: 10.1109/INTERCON.2018.8526375.

Jewel Kyle Fabula. How to play damath (with printable damath board and chips). *Career and Education*, 2022. URL `https://filipiknow.net/damath-board/#a-counting-damath`.

U. Gal. Damath notes. *Damath Seminar Workshop 1998*, 2017. URL `https://www.slideshare.net/slideshow/damath-notes-2/83625311#25`.

R. Gallotta, G. Todd, M. Zammit1, S. Earle, A. Liapis1, J. Togelius, and G. Yannakakis. Large language models and games: A survey and roadmap. *Computation and Language (cs.CL)*, 2024. doi: https://doi.org/10.48550/arXiv.2402.18659.

S. Gehrmann, H. Strobelt, and A. M. Rush. Gltr: Statistical detection and visualization of generated text. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2019. doi: https://doi.org/10.48550/arXiv.1906.04043.

Miguel Grinberg. *Flask web development: developing web applications with python*. " O'Reilly Media, Inc.", 2018.

Hongyi Guo, Zhihan Liu, Yufeng Zhang, and Zhaoran Wang. Can large language models play games? a case study of a self-play approach. 2024. URL `https://arxiv.org/pdf/2403.05632`.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024. URL `https://arxiv.org/abs/2308.00352`.

Shiying Hu, Zengrong Huang, Chengpeng Hu, and Jialin Liu. 3d building generation in minecraft via large language models. 2024. doi: 2406.08751. URL `https://arxiv.org/abs/2406.08751`.

K K Idzham, M W N Khalishah, Y W Steven, M S M F Aminuddin, H N Syawani, AM Zain, and Y Yusoff. Study of artificial intelligence into checkers game using html and javascript. *IOP Conference Series: Materials Science and Engineering*, 864(1):012091, may 2020. doi: 10.1088/1757-899X/864/1/012091. URL `https://dx.doi.org/10.1088/1757-899X/864/1/012091`.

Muhammad Sakib Khan Inan, Rizwan Hasan, and Tabia Tanzin Prama. An integrated expert system with a supervised machine learning based probabilistic approach to play tic-tac-toe. In *2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 0116–0120. IEEE, 2021.

Takumi Ito, Tatsuki Kuribayashi, Masatoshi Hidaka, Jun Suzuki, and Kentaro Inui. Langsmith: An interactive academic text revision system, 2020. URL `https://arxiv.org/abs/2010.04332`.

Sunil Karamchandani, Parth Gandhi, Omkar Pawar, and Shruti Pawaskar. A simple algorithm for designing an artificial intelligence based tic tac toe game. In *2015 International Conference on Pervasive Computing (ICPC)*, pages 1–4, 2015. doi: 10.1109/PERVASIVE.2015.7087182.

Wolfgang Konen. General board game playing for education and research in generic ai game learning. In *2019 IEEE Conference on Games (CoG)*, pages 1–8, 2019. doi: 10.1109/CIG.2019.8848070.

N. Koul. *Prompt Engineering for Large Language Models*. Nimrita Koul, 2023. ISBN 9789360130398. URL `https://books.google.com.ph/books?id=e9_jEAAAQBAJ`.

Vikram Kumaran, Jonathan Rowe, Bradford Mott, and James Lester. Scenecraft: Automating interactive narrative scene generation in digital games with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 19(1):86–96, 2023. doi: 10.1609/aiide.v19i1.27504. URL `https://ojs.aaai.org/index.php/AIIDE/article/view/27504`.

Pier Luca Lanzi and Daniele Loiacono. Chatgpt and other large language models as evolutionary engines for online interactive collaborative game design. *Proceedings of the Genetic and Evolutionary Computation Conference*, 2023. doi: 10.1145/3583131.3590351. URL `http://dx.doi.org/10.1145/3583131.3590351`.

J. Lee, W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So, and J Kang. Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 2020. doi: https://doi.org/10.48550/arXiv.1901.08746.

Pia Lee-Brago. Deped promotes “damath". *Philstar*, 2010. URL `www.philstar.com/headlines/2010/02/20/550971/deped-promotes-damath`.

Kenneth Li, Aspen K. Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. 2024. doi: 2210.13382. URL `https://arxiv.org/abs/2210.13382`.

Sai Ho Ling and Hak Keung Lam. Playing tic-tac-toe using genetic neural network with double transfer functions. *Journal of Intelligent Learning Systems and Applications*, 3(01):37–44, 2011.

Andrea Martinenghi, Gregor Donabauer, Simona Amenta, Sathya Bursic, Mathyas Giudici, Udo Kruschwitz, Franca Garzotto, and Dimitri Ognibene. Llms of catan: Exploring pragmatic capabilities of generative chatbots through prediction and classification of dialogue acts in boardgames' multi-party dialogues. 2024. URL `https://aclanthology.org/2024.games-1.12.pdf`.

Math and Multimedia. Introduction to the damath board game part 1. *Games and Puzzles, High School Mathematics*, 2016. URL `http://mathandmultimedia.com/2016/11/23/damath-1/`.

Malik Maulana. A neural network-based learning agent for the game of damath. Unpublished: School of Graduate Studies, Mindanao State University - Iligan Institute of Technology, 2019.

Vasilios Mavroudis. LangChain v0.3. working paper or preprint, December 2024. URL `https://hal.science/hal-04817573`.

A. Mentzelopoulou. Reducing LLM Hallucinations with Retrieval Prompt Engineering | TU Delft Repository — repository.tudelft.nl. `https://repository.tudelft.nl/record/uuid:c5176a21-c793-4eea-8893-87d151b8bfe7`, 2024. [Accessed 30-04-2025].

Merriam-Webster. Board game definition and meaning. `https://www.merriam-webster.com/dictionary/board%20game`. Accessed 04-07-2024.

Meta. GitHub - meta-llama/llama-prompt-ops: An open-source tool for seamless migration from other LLMs to Llama, and for general prompt optimization. — github.com. `https://github.com/meta-llama/llama-prompt-ops`, 2025. [Accessed 19-05-2025].

S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao. Large language models: A survey. *Computation and Language (cs.CL)*, pages 1–2, 2024. doi: https://doi.org/10.48550/arxiv.2402.06196.

David E Moriarty, Risto Miikkulainen, et al. Discovering complex othello strategies through evolutionary neural networks. *Connection Science*, 7(3):195–210, 1995.

Muhammad U. Nasir, Steven James, and Julian Togelius. Word2world: Generating stories and worlds through large language models. 2024. doi: 2405.06686. URL `https://arxiv.org/abs/2405.06686`.

H. Naveed, A.U. Khana, S. Qiub, M. Saqibc, S. Anware, M. Usmane, N. Akhtarg, N. Barnesh, and A. Miani. A comprehensive overview of large language models. *Computation and Language (cs.CL)*, 2023. doi: https://doi.org/10.48550/arXiv.2307.06435.

David Noever, Matthew Ciolino, and Josh Kalin. The chess transformer: Mastering play using generative language models. 2020. URL `https://arxiv.org/pdf/2008.04057`.

Farin Hanifatun Nuha and Kartika Yuni Purwanti. The effect of game based learning assisted by fun card puzzle on the conceptual understanding of class 5th elementary school students. *International Journal of Scientific Multidisciplinary Research*, 2023. doi: https://journal.formosapublisher.org/index.php/ijsmr/article/view/4754.

OpenAI, J. Achiam, S. Adler, S. Agarwal, and et. al. Gpt-4 technical report. *Computation and Language (cs.CL); Artificial Intelligence (cs.AI)*, 2023. doi: https://doi.org/10.48550/arXiv.2303.08774.

D. Qiao, C. Wu, Y. Liang, J. Li, and N. Duan. Gameeval: Evaluating llms on conversational games. *Computation and Language (cs.CL)*, 2023. doi: https://doi.org/10.48550/arXiv.2308.10032.

Shayryl Mae L Ramos, Izza G Legaspi, Joseph Anthony C Sabal, and Gerardo S Doroja. Mobile damath: a game for basic numeracy exercise. *International Journal of Arts and Technology*, 6(3):246–254, 2013.

Rosenfield Ronald. Two decades of statistical language modeling: Where do we go from here? *IEEE Conference Proceedings*, 2001. URL `https://www.cs.cmu.edu/~roni/papers/survey-slm-IEEE-PROC-0004.pdf`.

Ashutosh Kumar Sahu, Parthasarathi Palita, Anupam Mohanty, and Siksha O ITER. Tic-tac-toe game between computers: a computational intelligence approach. In *Proceedings of International conference on Frontiers in Intelligent Computing: Theory and Applications. ITER, SO, Odissa, India*, 2012.

Salimata Sawadogo, Aminata Sabané, and Rodrique Kafando andTegawendé F. Bissyandé. Revisiting the Non-Determinism of Code Generation by the GPT-3.5 Large Language Model (SANER 2025 - Reproducibility Studies and Negative Results (RENE) Track) - SANER 2025 — conf.researchr.org. `https://conf.researchr.org/details/saner-2025/saner-2025-reproducibility-studies-and-negative-results-rene-track/2/Revisiting-the-Non-Determinism-of-Code-Generation-by-the-GPT-3-5-Large-Language-Model?utm_source=perplexity`, 2025. [Accessed 08-05-2025].

Jonathan Schaeffer, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. Checkers is solved. *Science*, 317:1518–1522, 10 2007. doi: 10.1126/science.1144079.

H. Shum, X. He, and D. Li. From eliza to xiaoice: Challenges and opportunities with social chatbots. *Frontiers of Information Technology & Electronic Engineering*, 2018. doi: https://doi.org/10.48550/arXiv.1801.01957.

Michał Skibiński. GitHub - miskibin/py-draughts: A checkers library with modern web GUI, Supprots multiple variants of game. — github.com. `https://github.com/miskibin/py-draughts`, 2023. [Accessed 07-05-2025].

Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models. 2023. doi: 2302.05981. URL https://arxiv.org/abs/2302.05981.

Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (pcgml). 2018. doi: 1702.00539. URL https://arxiv.org/abs/1702.00539.

Pittawat Taveekitworachai, Febri Abdullah, Mury F. Dewantoro, Ruck Thawonmas, Julian Togelius, and Jochen Renz. Chatgpt4pcg competition: Character-like level generation for science birds. 2024. doi: 2303.15662. URL https://arxiv.org/abs/2303.15662.

Graham Todd, Sam Earle, Muhammad Umair Nasir, Michael Cerny Green, and Julian Togelius. Level generation through large language models. 2023. doi: 10.1145/3582437.3587211. URL http://dx.doi.org/10.1145/3582437.3587211.

Oguzhan Topsakal and Jackson B. Harper. Benchmarking large language model (llm) performance for game playing via tic-tac-toe. *Article in Electronics*, 2024. URL https://www.researchgate.net/publication/379891909_Benchmarking_Large_Language_Model_LLM_Performance_for_Game_Playing_via_Tic-Tac-Toe.

Shubham Toshniwal, Sam Wiseman, Karen Livescu, and Kevin Gimpel. Chess as a testbed for language model state tracking. 2021. URL https://arxiv.org/pdf/2102.13249.

Chen Feng Tsai, Xiaochen Zhou, Sierra S. Liu, Jing Li, Mo Yu, and Hongyuan Mei. Can large language models play text games well? current state-of-the-art and open questions. 2023. doi: 2304.02868. URL https://arxiv.org/abs/2304.02868.

Wikipedia. Damath, 2024. URL https://en.wikipedia.org/wiki/Damath.

Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, and J. Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *Computation and Language (cs.CL)*, 2016. doi: https://doi.org/10.48550/arXiv.1609.08144.

Yuzhuang Xu, Shuo Wang, Peng Li, Fuwen Luo, Xiaolong Wang, Weidong Liu, and Yang Liu. Exploring large language models for communication games: An empirical study on werewolf. 2024. doi: 2309.04658. URL https://arxiv.org/abs/2309.04658.

AM Zain, CW Chai, CC Goh, BJ Lim, CJ Low, and SJ Tan. Development of tic-tac-toe game using heuristic search. *IOP Conference Series: Materials Science and Engineering*, 864(1):012090, may 2020. doi: 10.1088/1757-899X/864/1/012090. URL https://dx.doi.org/10.1088/1757-899X/864/1/012090.

Yunlong Zhao, Chengpeng Hu, and Jialin Liu. Game generation via large language models. 2024. doi: 2404.08706. URL https://arxiv.org/abs/2404.08706.

Vytautas Štuikys and Robertas Damaševičius. Meta-programming and model-driven meta-program development. 2024. doi: 10.1007/978-1-4471-4126-6. URL books.google.com.ph/books?hl=en&lr=&id=B85FcaE9YdgC&oi=fnd&pg=PR5&dq=meta+programming&ots=9pdVyjuZ0W&sig=sscm-3uTYV_H_Z3xiRMOf3hz64M&redir_esc=y#v=onepage&q=meta%20programming&f=false.